

Processos Cognitivos, Redes Neurais & Matrizes

Prof. Dr. Leonardo Lana de Carvalho

Professor Associado UFVJM – Departamento de Computação - DECOM.

Orientador no Mestrado em Ciências Humanas - MPICH

Email: leonardolana.carvalho@ufvjm.edu.br

Sumário

1. Matrizes & Vetores
2. Memória auto-associativa com aprendizagem hebbiana
3. Memória auto-associativa com aprendizagem de Widrow-Hoff
4. Memória linear hetero-associativa
5. ADALINE (*Adaptive Linear Neuron*)
6. ADALINE logística
7. Perceptron, Funções Lógicas e o problema XOR
8. Perceptron Multicamadas & Funções Lógicas

Matrizes & Vetores

- Matriz de Produtos Cruzados
 - Multiplica-se uma matriz por sua transposta;
 - Por definição ela é quadrada e simétrica

$$A = \begin{bmatrix} 1 & 1 \\ 2 & 4 \\ 3 & 4 \end{bmatrix}$$

$$A^T A = \begin{bmatrix} 1 * 1 + 2 * 2 + 3 * 3 & 1 * 1 + 2 * 4 + 3 * 4 \\ 1 * 1 + 4 * 2 + 4 * 3 & 1 * 1 + 4 * 4 + 4 * 4 \end{bmatrix}$$

$$A^T = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 4 & 4 \end{bmatrix}$$

$$A^T * A = \begin{bmatrix} 14 & 21 \\ 21 & 33 \end{bmatrix}$$

Matrizes & Vetores

- Produto Externo de Dois Vetores
 - Podemos associar uma matriz a qualquer casal de vetores pela operação ab^T

$$a * b^T = \begin{bmatrix} a_1 * b_1 & \dots & a_1 * b_j & \dots & a_1 * b_J \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ a_i * b_1 & \dots & a_i * b_j & \dots & a_i * b_J \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ a_I * b_1 & \dots & a_I * b_j & \dots & a_I * b_J \end{bmatrix}$$

Matrizes & Vetores

- Produto Externo de Dois Vetores
 - Podemos associar uma matriz a qualquer casal de vetores pela operação ab^T

$$a = \begin{bmatrix} 2 \\ 5 \\ 10 \\ 20 \end{bmatrix} \quad b = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad b^T = [1 \quad 2 \quad 3]$$

$$a \times b^T = \begin{bmatrix} 2 & 4 & 6 \\ 5 & 10 & 15 \\ 10 & 20 & 30 \\ 20 & 40 & 60 \end{bmatrix}$$

Redes Neurais Artificiais: Memória Auto-associativa

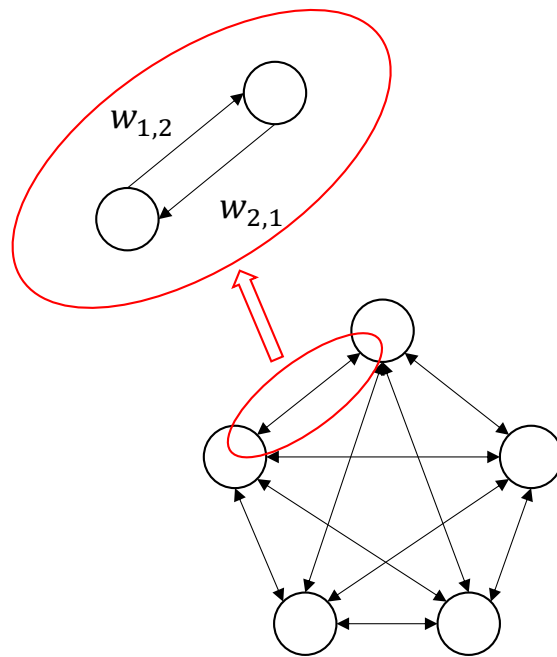
1. Quando um padrão familiar vem à memória, ele é ampliado e a resposta é forte;
2. Quando um padrão desconhecido chega na memória a atividade é reduzida;
3. Quando apenas uma parte do padrão externo familiar chega à memória, a memória pode ainda responder através de um processo de reconstrução;
4. Quando um padrão semelhante a um padrão familiar vem à memória, ele responde por distorção, não procede pela diferenciação e responde como se aquele fosse o padrão familiar;
5. Se padrões múltiplos são armazenados, o sistema de memória irá responder com uma tendência central a um padrão de entrada, seja ele novo ou familiar. A memória cria um protótipo, mesmo que este padrão do protótipo nunca tenha se apresentado antes.
6. Uma rede neural artificial nem sempre faz generalizações corretas.



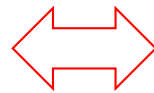
Zhang (2007)

Rumelhart (1998)

Memória auto-associativa com aprendizagem hebbiana



Memória
auto-associativa



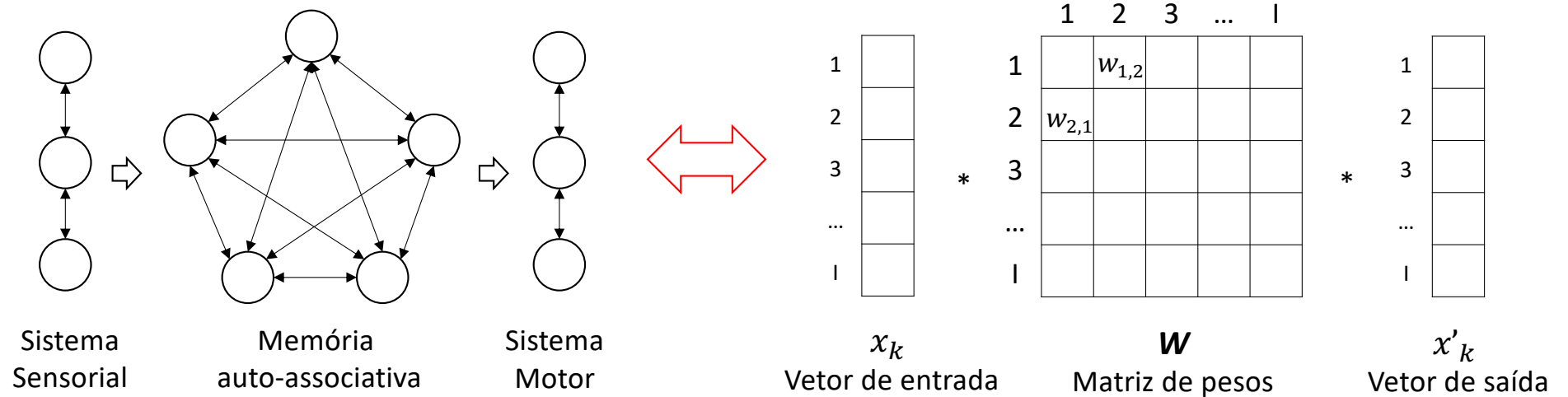
	1	2	3	...	I
1		$w_{1,2}$			
2	$w_{2,1}$				
3					
...					
I					

W

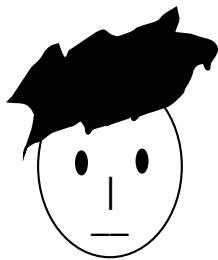
Matriz de pesos

Donald Hebb (1904-1985)
Neuropsicólogo canadense. Para Hebb a psicologia não pode ser investigada senão integrando o estudo da cognição com o da neurofisiologia. Suas teorias sobre a aprendizagem fundam as neurociências cognitivas.

Memória auto-associativa com aprendizagem hebbiana



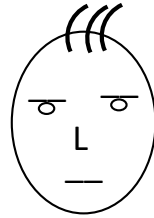
Memória auto-associativa com aprendizagem hebbiana



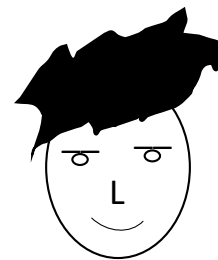
+1+1+1-1
Rosto 1



-1-1+1-1
Rosto 2



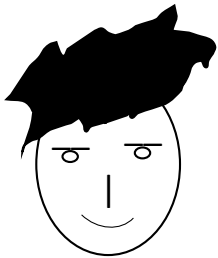
-1-1-1-1
Rosto 3



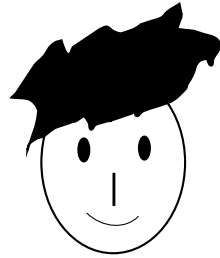
+1-1-1+1
Rosto 4



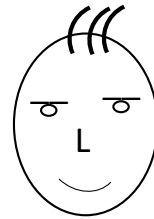
-1+1-1-1
Rosto 5



+1-1+1+1
Rosto 6



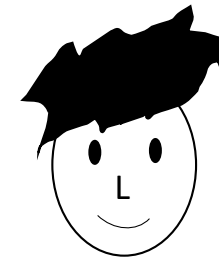
+1+1+1+1
Rosto 7



-1-1-1+1
Rosto 8



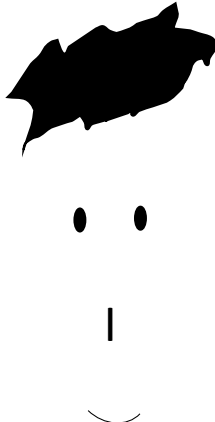
-1+1+1-1
Rosto 9



+1+1-1+1
Rosto 10

Memória auto-associativa com aprendizagem hebbiana

Tabela: Conjunto de traços utilizados para a criação de rostos

Células	Traço	+1	-1
1	Cabelo		
2	Olhos		
3	Nariz		
4	Boca		

Memória auto-associativa com aprendizagem hebbiana

A **regra de Hebb** estipula que a intensidade (o peso) da conexão entre duas células i e j é proporcional à ativação das duas células:

$$w_{i,j} = \gamma(x_i * x_j)$$

Tomemos a constante de proporcionalidade $\gamma = 1$.

- O conjunto dos estímulos podem ser assimilados (estocados) na memória generalizando a regra de Hebb assim:

$$W = \sum_{k=1}^K x_k x_k^T = X X^T$$

Memória auto-associativa com aprendizagem hebbiana

- O algoritmo de aprendizagem de Hebb inicia com as conexões entre as células com valores nulos.

$$W_{[0]} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Memória auto-associativa com aprendizagem hebbiana

Na primeira etapa, o primeiro rosto é estocado na memória:

$$\begin{aligned} W_{[1]} &= W_{[0]} + x_1 x_1^T \\ &= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} * [1 \quad 1 \quad 1 \quad -1] \\ &= \begin{bmatrix} 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ -1 & -1 & -1 & 1 \end{bmatrix} \end{aligned}$$

Memória auto-associativa com aprendizagem hebbiana

Na segunda etapa, o segundo rosto é estocado na memória:

$$\begin{aligned} W_{[2]} &= W_{[1]} + x_2 x_2^T \\ &= \begin{bmatrix} 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ -1 & -1 & -1 & 1 \end{bmatrix} + \begin{bmatrix} -1 \\ -1 \\ 1 \\ -1 \end{bmatrix} * [-1 \quad -1 \quad 1 \quad -1] \\ &= \begin{bmatrix} 2 & 2 & 0 & 0 \\ 2 & 2 & 0 & 0 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & -2 & 2 \end{bmatrix} \end{aligned}$$

Memória auto-associativa com aprendizagem hebbiana

Assim até a décima etapa, quando teremos:

$$\begin{aligned} W_{[10]} &= W_{[9]} + x_{10}x_{10}^T \\ &= \begin{bmatrix} 9 & 1 & 3 & 5 \\ 1 & 9 & 3 & -3 \\ 3 & 3 & 9 & -1 \\ 5 & -3 & -1 & 9 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ -1 \\ 1 \end{bmatrix} * [1 \quad 1 \quad -1 \quad 1] \\ &= \begin{bmatrix} 10 & 2 & 2 & 6 \\ 2 & 10 & 2 & -2 \\ 2 & 2 & 10 & -2 \\ 6 & -2 & -2 & 10 \end{bmatrix} \end{aligned}$$

Memória auto-associativa com aprendizagem hebbiana

Verifica-se que $W_{[10]}$ é igual ao produto cruzado da matriz X de ordem 4x10 representando os 10 rostos.

$$\begin{aligned} W_{[10]} &= XX^T \\ &= \begin{bmatrix} +1 & -1 & -1 & +1 & -1 & +1 & +1 & -1 & -1 & +1 \\ +1 & -1 & -1 & -1 & +1 & -1 & +1 & -1 & +1 & +1 \\ +1 & +1 & -1 & -1 & -1 & +1 & +1 & -1 & +1 & -1 \\ -1 & -1 & -1 & +1 & -1 & +1 & +1 & +1 & -1 & +1 \end{bmatrix} * \begin{bmatrix} +1 & +1 & +1 & -1 \\ -1 & -1 & +1 & -1 \\ -1 & -1 & -1 & -1 \\ +1 & -1 & -1 & +1 \\ -1 & +1 & -1 & -1 \\ +1 & -1 & +1 & +1 \\ +1 & +1 & +1 & +1 \\ -1 & -1 & -1 & +1 \\ -1 & +1 & +1 & -1 \\ +1 & +1 & -1 & +1 \end{bmatrix} \\ &= \begin{bmatrix} 10 & 2 & 2 & 6 \\ 2 & 10 & 2 & -2 \\ 2 & 2 & 10 & -2 \\ 6 & -2 & -2 & 10 \end{bmatrix} \end{aligned}$$

Memória auto-associativa com aprendizagem hebbiana

A **lembrança** de um rosto pela memória obtém-se pela multiplicação do vetor correspondente ao rosto x_k com a matriz de conexões W .

$$x'_1 = Wx_1$$

$$= \begin{bmatrix} 10 & 2 & 2 & 6 \\ 2 & 10 & 2 & -2 \\ 2 & 2 & 10 & -2 \\ 6 & -2 & -2 & 10 \end{bmatrix} * \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} = \begin{bmatrix} (10 * 1) + (2 * 1) + (2 * 1) + (6 * -1) \\ (2 * 1) + (10 * 1) + (2 * 1) + (-2 * -1) \\ (2 * 1) + (2 * 1) + (10 * 1) + (-2 * -1) \\ (6 * 1) + (-2 * 1) + (-2 * 1) + (10 * -1) \end{bmatrix}$$

$$= \begin{bmatrix} 8 \\ 16 \\ 16 \\ -8 \end{bmatrix}$$

Memória auto-associativa com aprendizagem hebbiana

A qualidade da resposta x'_1 é geralmente avaliada pelo cosseno entre x_1 e x'_1 . Diremos que quanto mais próximo de 1 é o cosseno mais os dois vetores se assemelham.

$$\cos(x'_1, x_1) = \frac{x'^T_1 x_1}{\|x'_1\| \|x_1\|}$$

$$\begin{aligned}\|x'_1\| &= \sqrt{x'^T_1 x'_1} \\ &= \sqrt{640} \\ &= 25.2982\end{aligned}$$

$$\begin{aligned}\|x_1\| &= \sqrt{x^T_1 x_1} \\ &= \sqrt{4} \\ &= 2\end{aligned}$$

$$\begin{aligned}x'^T_1 x_1 &= [8 \quad 16 \quad 16 \quad -8] \times \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} \\ &= 8+16+16+8 \\ &= 48\end{aligned}$$

Memória auto-associativa com aprendizagem hebbiana

O valor de $\cos(x'_1, x_1)$ sendo diferente de 1, mas próximo, diz que a resposta dada pela memória não é perfeita mas, de um modo satisfatório, semelhante.

$$\begin{aligned}\cos(x'_1, x_1) &= \frac{x'^T_1 x_1}{\|x'_1\| \|x_1\|} \\ &= \frac{48}{25.2982 * 2} \\ &= \frac{48}{50.5964} \\ &= 0.9487\end{aligned}$$

Memória auto-associativa com aprendizagem hebbiana

O conjunto de rostos aprendidos pode ser lembrado em uma só etapa multiplicando X pela matriz de conexões W .

$$X' = WX$$

$$= \begin{bmatrix} 10 & 2 & 2 & 6 \\ 2 & 10 & 2 & -2 \\ 2 & 2 & 10 & -2 \\ 6 & -2 & -2 & 10 \end{bmatrix} * \begin{bmatrix} +1 & -1 & -1 & +1 & -1 & +1 & +1 & -1 & -1 & +1 \\ +1 & -1 & -1 & -1 & +1 & -1 & +1 & -1 & +1 & +1 \\ +1 & +1 & -1 & -1 & -1 & +1 & +1 & -1 & +1 & -1 \\ -1 & -1 & -1 & +1 & -1 & +1 & +1 & +1 & -1 & +1 \end{bmatrix}$$

$$= \begin{bmatrix} +8 & -16 & -20 & +12 & -16 & +16 & +20 & -8 & -12 & +16 \\ +16 & -9 & -13 & -11 & +7 & -7 & +13 & -15 & +11 & +9 \\ +16 & +8 & -12 & -12 & -8 & +8 & +12 & -16 & +12 & -8 \\ -8 & -16 & -12 & +20 & -16 & +16 & +12 & +8 & -20 & +16 \end{bmatrix}$$

Recapitulando o formulário...

$$W_{[i]} = XX^T \quad x'_1 = Wx_1$$

$$\cos(x'_1, x_1) = \frac{x'^T_1 x_1}{\|x'_1\| \|x_1\|}$$

$$\|x'_1\| = \sqrt{x'^T_1 x'_1} \quad \|x_1\| = \sqrt{x^T_1 x_1}$$

Inicializando matrizes bidimensionais

$$W_{[0]} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

[Exemplo 4]

```
#include <iostream>
using namespace std;

int main (){
    double W[4][4]={
        {0,0,0,0},
        {0,0,0,0},
        {0,0,0,0},
        {0,0,0,0}
    };
    for (int i=0; i<4; i++){
        for (int j=0; j<4; j++){
            cout << W[i][j] << " ";
        }
        cout << endl;
    }
}
```

Produto Externo de dois Vetores

$$x_1 x_1^T = \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} * [1 \quad 1 \quad 1 \quad -1]$$

$$= \begin{bmatrix} 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ -1 & -1 & -1 & 1 \end{bmatrix}$$

Produto Externo de dois Vetores

```
#include <stdio.h>
#include <iostream>
#include <iomanip>
using namespace std;
#define I 4
#define J 4
```

```
double X[I] = {1,1,1,-1};
double W[I][J] = {
    {0,0,0,0},
    {0,0,0,0},
    {0,0,0,0},
    {0,0,0,0}
};
```

```
void produtoExterno (){
    int i,j;
    for (i=0; i<I; i++){
        for (j=0; j<J; j++){
            W[i][j]=X[i]*X[j];
            cout << setprecision (4)
                  << setw (8)
                  << W[i][j];
        }
        cout << endl;
    }
}

int main () {
    produtoExterno();
}
```



```

#include <iostream>
#include <iomanip>
using namespace std;
#define I 4
#define J 4

void produtoExterno (double W[][J], double X[]){
    int i,j;
    for (i=0; i<I; i++){
        for (j=0; j<J; j++){
            W[i][j]=X[i]*X[j];
            cout << setprecision (4) << setw (8)
                << W[i][j];
        }
        cout << endl;
    }
}

```

```

int main () {
    double X[I] = {1,1,1,-1};
    double W[I][J] = {
        {0,0,0,0},
        {0,0,0,0},
        {0,0,0,0},
        {0,0,0,0}
    };
    produtoExterno(W,X);
}

```

[Exemplo 6]

Produto Cruzado

$$W_{[10]} = XX^T$$

$$= \begin{bmatrix} +1 & -1 & -1 & +1 & -1 & +1 & +1 & -1 & -1 & +1 \\ +1 & -1 & -1 & -1 & +1 & -1 & +1 & -1 & +1 & +1 \\ +1 & +1 & -1 & -1 & -1 & +1 & +1 & -1 & +1 & -1 \\ -1 & -1 & -1 & +1 & -1 & +1 & +1 & +1 & -1 & +1 \end{bmatrix} * \begin{bmatrix} +1 & +1 & +1 & -1 \\ -1 & -1 & +1 & -1 \\ -1 & -1 & -1 & -1 \\ +1 & -1 & -1 & +1 \\ -1 & +1 & -1 & -1 \\ +1 & -1 & +1 & +1 \\ +1 & +1 & +1 & +1 \\ -1 & -1 & -1 & +1 \\ -1 & +1 & +1 & -1 \\ +1 & +1 & -1 & +1 \end{bmatrix}$$

$$= \begin{bmatrix} 10 & 2 & 2 & 6 \\ 2 & 10 & 2 & -2 \\ 2 & 2 & 10 & -2 \\ 6 & -2 & -2 & 10 \end{bmatrix}$$

```
#include <iostream>
#include <iomanip>
using namespace std;
```

```
double W[4][4];
double X[4][10]={
    { 1,-1,-1, 1,-1, 1, 1,-1,-1, 1},
    { 1,-1,-1,-1, 1,-1, 1,-1, 1, 1},
    { 1, 1,-1,-1,-1, 1, 1,-1, 1,-1},
    {-1,-1,-1, 1,-1, 1, 1, 1,-1, 1}
};
```

```
int main(){
    for (int i=0; i<4; i++){
        for (int j=0; j<4; j++){
            W[i][j]=0;
            for (int k=0; k<10; k++){
                W[i][j]=W[i][j]+X[i][k]*X[j][k];
            }
        }
    }
    for (int i=0; i<4; i++){
        for (int j=0; j<4; j++){
            cout << setprecision(4)
                << setw(8) << W[i][j];
        }
        cout << endl;
    }
    cout << endl;
}
```

[Exemplo 8]

Produto Cruzado

$$X' = WX$$

$$= \begin{bmatrix} 10 & 2 & 2 & 6 \\ 2 & 10 & 2 & -2 \\ 2 & 2 & 10 & -2 \\ 6 & -2 & -2 & 10 \end{bmatrix} * \begin{bmatrix} +1 & -1 & -1 & +1 & -1 & +1 & +1 & -1 & -1 & +1 \\ +1 & -1 & -1 & -1 & +1 & -1 & +1 & -1 & +1 & +1 \\ +1 & +1 & -1 & -1 & -1 & +1 & +1 & -1 & +1 & -1 \\ -1 & -1 & -1 & +1 & -1 & +1 & +1 & +1 & -1 & +1 \end{bmatrix}$$

$$= \begin{bmatrix} +8 & -16 & -20 & +12 & -16 & +16 & +20 & -8 & -12 & +16 \\ +16 & -8 & -12 & -12 & +8 & -8 & +12 & -16 & +12 & +8 \\ +16 & +8 & -12 & -12 & -8 & +8 & +12 & -16 & +12 & -8 \\ -8 & -16 & -12 & +20 & -16 & +16 & +12 & +8 & -20 & +16 \end{bmatrix}$$

```

#include <iostream>
#include <iomanip>
using namespace std;

int main(){
    double W[4][4]={
        {10, 2, 2, 6},
        { 2,10, 2,-2},
        { 2, 2,10,-2},
        { 6,-2,-2,10}
    };
    double O[4][10];
    double X[4][10]={
        { 1,-1,-1, 1,-1, 1,1,-1,-1, 1},
        { 1,-1,-1,-1, 1,-1,1,-1, 1, 1},
        { 1, 1,-1,-1,-1, 1,1,-1, 1,-1},
        {-1,-1,-1, 1,-1, 1,1, 1,-1, 1}
    };
    for (int i=0; i<4; i++){
        for (int k=0; k<10; k++){
            for (int j=0; j<4; j++){
                O[i][k]=O[i][k]+W[i][j]*X[j][k];
            }
        }
    }
    for (int i=0; i<4; i++){
        for (int k=0; k<10; k++){
            cout << setprecision(4) << setw(8) << O[i][k];
        }
        cout << endl;
    }
}

```

[Exemplo 9]

Cosseno de dois vetores

$$\cos(x'_1, x_1) = \frac{x'^T_1 x_1}{\|x'_1\| \|x_1\|}$$

$$x'^T_1 x_1$$

$$= \begin{bmatrix} +8 & -16 & -20 & +12 & -16 & +16 & +20 & -8 & -12 & +16 \\ +16 & -8 & -12 & -12 & +8 & -8 & +12 & -16 & +12 & +8 \\ +16 & +8 & -12 & -12 & -8 & +8 & +12 & -16 & +12 & -8 \\ -8 & -16 & -12 & +20 & -16 & +16 & +12 & +8 & -20 & +16 \end{bmatrix}$$

$$* \begin{bmatrix} +1 & -1 & -1 & +1 & -1 & +1 & +1 & -1 & -1 & +1 \\ +1 & -1 & -1 & -1 & +1 & -1 & +1 & -1 & +1 & +1 \\ +1 & +1 & -1 & -1 & -1 & +1 & +1 & -1 & +1 & -1 \\ -1 & -1 & -1 & +1 & -1 & +1 & +1 & +1 & -1 & +1 \end{bmatrix}$$

$$= [48 \quad 48 \quad 56 \quad 56 \quad 48 \quad 48 \quad 56 \quad 48 \quad 56 \quad 48]$$

```
#include <iostream>
#include <iomanip>
using namespace std;
```

```
int main(){
    double X[4][10]={
        { 1,-1,-1, 1,-1, 1,1,-1,-1, 1},
        { 1,-1,-1,-1, 1,-1,1,-1, 1, 1},
        { 1, 1,-1,-1,-1, 1,1,-1, 1,-1},
        {-1,-1,-1, 1,-1, 1,1, 1,-1, 1}
    };
    double O[4][10]={
        { 8,-16,-20, 12,-16,16,20, -8,-12,16},
        {16, -8,-12,-12, 8,-8,12,-16, 12, 8},
        {16, 8,-12,-12, -8, 8,12,-16, 12,-8},
        {-8,-16,-12, 20,-16,16,12, 8,-20,16}
    };
};
```

```

#include <iostream>
#include <iomanip>
using namespace std;

int main(){
    double X[4][10]={
        { 1,-1,-1, 1,-1, 1,1,-1,-1, 1},
        { 1,-1,-1,-1, 1,-1,1,-1, 1, 1},
        { 1, 1,-1,-1,-1, 1,1,-1, 1,-1},
        {-1,-1,-1, 1,-1, 1,1, 1,-1, 1}
    };
    double O[4][10]={
        { 8,-16,-20, 12,-16,16,20, -8,-12,16},
        {16, -8,-12,-12, 8,-8,12,-16, 12, 8},
        {16, 8,-12,-12, -8, 8,12,-16, 12,-8},
        {-8,-16,-12, 20,-16,16,12, 8,-20,16}
    };

double cosseno[10];
for (int i=0; i<10; i++){
    cosseno[i]=0;
}
for (int j=0; j<10; j++){
    for (int i=0; i<4; i++){
        cosseno[j]=coosseno[j]+O[i][j]*X[i][j];
        //cout << setprecision(4) << setw(8) << cosseno[j];
    }
    //cout << endl;
}
for (int i=0; i<10; i++){
    cout << setprecision(4) << setw(8) << cosseno[i];
}
cout << endl;
}

```

[Exemplo 10]

Memória auto-associativa com aprendizagem de Widrow-Hoff

- A regra de aprendizagem de Hebb conduz a performances que não são perfeitas: os rostos obtidos pela resposta da memória estão deformados.
- O cosseno entre os vetores, o de entrada e o de reconstrução da imagem pela memória, não é igual a 1.
- A regra de aprendizagem de Widrow-Hoff permite ajustar o peso das conexões entre as células da memória até que a **performance seja maximizada**.

Widrow, B., & Hoff, M. E. (1960). **Adaptive switching circuits**. IRE WESCON Convention Records, 96-104.

Memória auto-associativa com aprendizagem de Widrow-Hoff

1. Cálculo do Erro: Um estímulo é apresentado, em seguida é calculada a diferença da resposta esperada e da resposta apresentada.
2. Uma série de iterações é posta em prática para correção da matriz de pesos.
3. Se o erro é igual a 0 ou não decresce mais, então a estrutura de repetição se encerra.

Memória auto-associativa com aprendizagem de Widrow-Hoff

- Formalmente, a matriz de conexões para a iteração $n+1$ é dada por:

$$\begin{aligned}W_{[n+1]} &= W_{[n]} + \eta(x_k - W_{[n]}x_k)x_k^T \\&= W_{[n]} + \eta(x_k - x'_k)x_k^T \\&= W_{[n]} + \eta ex_k^T \\&= W_{[n]} + \Delta w_{[n]}\end{aligned}$$

η (“eta”) representa uma constante de aprendizagem

e é o vetor de erro

$\Delta w_{[n]}$ é a matriz de correção para a iteração n

k é o estímulo escolhido

Memória auto-associativa com aprendizagem de Widrow-Hoff

- Além do método por exemplar apontado anteriormente, podemos usar também uma aprendizagem por pacote (*batch learning*):

$$\begin{aligned}W_{[n+1]} &= W_{[n]} + \eta(X - W_{[n]}X)X^T \\&= W_{[n]} + \eta(X - X')X^T \\&= W_{[n]} + \eta EX^T \\&= W_{[n]} + \Delta w_{[n]}\end{aligned}$$

E é a matriz de erro

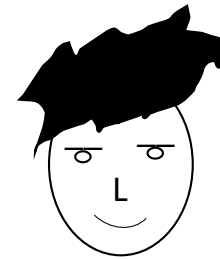
Memória auto-associativa com aprendizagem de Widrow-Hoff

- Exemplo
 - Aprendizagem de dois rostos pela rede de neurônios

$$X = \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix}$$



-1-1+1-1
Rosto 2



+1-1-1+1
Rosto 4

Memória auto-associativa com aprendizagem de Widrow-Hoff

- Inicializando a aprendizagem

$$\eta = 0,2$$

$$W_{[0]} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Memória auto-associativa com aprendizagem de Widrow-Hoff

- Inicializando a aprendizagem

$$\eta = 0,2$$

$$W_{[0]} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

```
#include <iostream>
#include <iomanip>
using namespace std;
```

```
double ETA = 0.2;
double X[4][2]={
    {-1, 1},
    {-1,-1},
    { 1,-1},
    {-1, 1}
};
double W[4][4]={
    {0,0,0,0},
    {0,0,0,0},
    {0,0,0,0},
    {0,0,0,0}
};
double O[4][2];
double E[4][2];
double DELTA[4][4];
```

Memória auto-associativa com aprendizagem de Widrow-Hoff

```
void zerandoMatrizes();
void resposta();
void calcularErro();
void geraDelta();
void corrigirW ();

int main (){
    for (int epoca=0; epoca<30; epoca++){
        cout << "-----" << " Iteração " << epoca+1 << " -----" << endl;
        zerandoMatrizes();
        resposta();
        calcularErro();
        geraDelta();
        corrigirW();
    }
}
```


Memória auto-associativa com aprendizagem de Widrow-Hoff

- Inicializando a aprendizagem

```
void zerandoMatrizes(){  
    for (int i=0; i<4; i++){  
        for (int j=0; j<2; j++){  
            O[i][j]=0;  
        }  
    }  
    for (int i=0; i<4; i++){  
        for (int j=0; j<4; j++){  
            DELTA[i][j]=0;  
        }  
    }  
}
```

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 1

- Etapa 1: Lembrar os rostos

$$X'_{[1]} = W_{[0]}X$$

$$= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 1

- Etapa 1: Lembrar os rostos

$$X'_{[1]} = W_{[0]}X$$

$$= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

```
void resposta(){
    cout << "Resposta/Erro/DeltaW\n"
          << endl;
    //Produto cruzado
    for (int i=0; i<4; i++){
        for (int k=0; k<2; k++){
            for (int j=0; j<4; j++){
                O[i][k]+=W[i][j]*X[j][k];
            }
        }
    }
    for (int i=0; i<4; i++){
        for (int k=0; k<2; k++){
            cout << setw(10) << fixed
                  << setprecision(5)
                  << O[i][k];
        }
        cout << endl;
    }
    cout << endl;
}
```

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 1

- Etapa 2: Calcular o erro

$$E_{[1]} = X - X'_{[1]}$$

$$= \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix}$$

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 1

- Etapa 2: Calcular o erro

$$E_{[1]} = X - X'_{[1]}$$

$$= \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix}$$

```
void calcularErro(){
    //Calcular o erro
    for (int i=0; i<4; i++){
        for (int k=0; k<2; k++){
            E[i][k]=X[i][k]-O[i][k];
            cout << setw(10)
                  << fixed
                  << setprecision(5)
                  << E[i][k];
        }
        cout << endl;
    }
    cout << endl;
}
```

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 1

- Etapa 3: Calcular a matriz de correção

$$\Delta w_{[1]} = 0,2 * E_{[1]} X^T$$

$$= 0,2 * \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix} * \begin{bmatrix} -1 & -1 & +1 & -1 \\ +1 & -1 & -1 & +1 \end{bmatrix}$$

$$= \begin{bmatrix} +0,4 & 0 & -0,4 & +0,4 \\ 0 & +0,4 & 0 & 0 \\ -0,4 & 0 & +0,4 & -0,4 \\ +0,4 & 0 & -0,4 & +0,4 \end{bmatrix}$$

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 1

- Etapa 3: Calcular a matriz de correção

$$\Delta w_{[1]} = 0,2 * E_{[1]} X^T$$

$$= 0,2 * \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix} * \begin{bmatrix} -1 & -1 & +1 & -1 \\ +1 & -1 & -1 & +1 \end{bmatrix}$$

$$= \begin{bmatrix} +0,4 & 0 & -0,4 & +0,4 \\ 0 & +0,4 & 0 & 0 \\ -0,4 & 0 & +0,4 & -0,4 \\ +0,4 & 0 & -0,4 & +0,4 \end{bmatrix}$$

```
void geraDelta(){
    //Calcular a matriz de correção
    for (int i=0; i<4; i++){
        for (int j=0; j<4; j++){
            for (int k=0; k<2; k++){
                DELTA[i][j]+=E[i][k]*X[j][k];
            }
        }
    }
    //Multiplica DELTA pela constante
    for (int i=0; i<4; i++){
        for (int j=0; j<4; j++){
            DELTA[i][j]*=ETA;
            cout << setw(10) << fixed
                << setprecision(5)
                << DELTA[i][j];
        }
        cout << endl;
    }
    cout << endl;
}
```

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 1

- Etapa 4: Corrigir a matriz de pesos

$$W_{[1]} = W_{[0]} + \Delta w_{[1]}$$

$$= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} +0,4 & 0 & -0,4 & +0,4 \\ 0 & +0,4 & 0 & 0 \\ -0,4 & 0 & +0,4 & -0,4 \\ +0,4 & 0 & -0,4 & +0,4 \end{bmatrix}$$

$$= \begin{bmatrix} +0,4 & 0 & -0,4 & +0,4 \\ 0 & +0,4 & 0 & 0 \\ -0,4 & 0 & +0,4 & -0,4 \\ +0,4 & 0 & -0,4 & +0,4 \end{bmatrix}$$

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 1

- Etapa 4: Corrigir a matriz de pesos

$$W_{[1]} = W_{[0]} + \Delta w_{[1]}$$

$$= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} +0,4 & 0 & -0,4 & +0,4 \\ 0 & +0,4 & 0 & 0 \\ -0,4 & 0 & +0,4 & -0,4 \\ +0,4 & 0 & -0,4 & +0,4 \end{bmatrix}$$

$$= \begin{bmatrix} +0,4 & 0 & -0,4 & +0,4 \\ 0 & +0,4 & 0 & 0 \\ -0,4 & 0 & +0,4 & -0,4 \\ +0,4 & 0 & -0,4 & +0,4 \end{bmatrix}$$

```
void corrigirW (){
    cout << "Matriz de pesos"
        << endl;
    for (int i=0; i<4; i++){
        for (int j=0; j<4; j++){
            W[i][j]+=DELTA[i][j];
            cout << setw(10)
                << fixed
                << setprecision(5)
                << W[i][j];
        }
        cout << endl;
    }
    cout << endl;
}
```

Memória auto-associativa com aprendizagem de Widrow-Hoff

```
void zerandoMatrizes();
void resposta();
void calcularErro();
void geraDelta();
void corrigirW ();

int main (){
    for (int epoca=0; epoca<30; epoca++){
        cout << "-----" << " Etapa " << epoca+1 << " -----" << endl;
        zerandoMatrizes();
        resposta();
        calcularErro();
        geraDelta();
        corrigirW();
    }
}
```

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 2

- Etapa 1: Lembrar os rostos

$$X'_{[2]} = W_{[1]}X$$

$$= \begin{bmatrix} +0,4 & 0 & -0,4 & +0,4 \\ 0 & +0,4 & 0 & 0 \\ -0,4 & 0 & +0,4 & -0,4 \\ +0,4 & 0 & -0,4 & +0,4 \end{bmatrix} * \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} -1,2 & +1,2 \\ -0,4 & -0,4 \\ +1,2 & -1,2 \\ -1,2 & +1,2 \end{bmatrix}$$

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 2

- Etapa 2: Calcular o erro

$$E_{[2]} = X - X'_{[2]}$$

$$= \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix} - \begin{bmatrix} -1,2 & +1,2 \\ -0,4 & -0,4 \\ +1,2 & -1,2 \\ -1,2 & +1,2 \end{bmatrix}$$

$$= \begin{bmatrix} +0,2 & -0,2 \\ -0,6 & -0,6 \\ -0,2 & +0,2 \\ +0,2 & -0,2 \end{bmatrix}$$

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 2

- Etapa 3: Calcular a matriz de correção

$$\Delta w_{[1]} = 0,2 * E_{[1]} X^T$$

$$= 0,2 * \begin{bmatrix} +0,2 & -0,2 \\ -0,6 & -0,6 \\ -0,2 & +0,2 \\ +0,2 & -0,2 \end{bmatrix} * \begin{bmatrix} -1 & -1 & +1 & -1 \\ +1 & -1 & -1 & +1 \end{bmatrix}$$

$$= \begin{bmatrix} -0,08 & 0 & +0,08 & -0,08 \\ 0 & +0,24 & 0 & 0 \\ +0,08 & 0 & -0,08 & +0,08 \\ -0,08 & 0 & +0,08 & -0,08 \end{bmatrix}$$

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 2

- Etapa 4: Corrigir a matriz de pesos

$$W_{[2]} = W_{[1]} + \Delta w_{[2]}$$

$$= \begin{bmatrix} +0,4 & 0 & -0,4 & +0,4 \\ 0 & +0,4 & 0 & 0 \\ -0,4 & 0 & +0,4 & -0,4 \\ +0,4 & 0 & -0,4 & +0,4 \end{bmatrix} + \begin{bmatrix} -0,08 & 0 & +0,08 & -0,08 \\ 0 & +0,24 & 0 & 0 \\ +0,08 & 0 & -0,08 & +0,08 \\ -0,08 & 0 & +0,08 & -0,08 \end{bmatrix}$$

$$= \begin{bmatrix} +0,32 & 0 & -0,32 & +0,32 \\ 0 & +0,64 & 0 & 0 \\ -0,32 & 0 & +0,32 & -0,32 \\ +0,32 & 0 & -0,32 & +0,32 \end{bmatrix}$$

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 24

- Etapa 1: Lembrar os rostos

$$X'_{[24]} = W_{[23]}X$$

$$\begin{aligned} &= \begin{bmatrix} +0,333 & 0 & -0,333 & +0,333 \\ 0 & 1 & 0 & 0 \\ -0,333 & 0 & +0,333 & -0,333 \\ +0,333 & 0 & -0,333 & +0,333 \end{bmatrix} * \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix} \\ &= \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix} \end{aligned}$$

Após repetirmos estes 4 passos um total de 24 vezes, o estímulo (pacote de dois rostos) é perfeitamente reconstruído. Teremos assim um **erro de reconstrução nulo**.

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 24

- Etapa 2: Calcular o erro

$$E_{[24]} = X - X'_{[24]}$$

$$= \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix} - \begin{bmatrix} -1 & 1 \\ -1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 24

- Etapa 3: Calcular a matriz de correção

$$\Delta w_{[24]} = 0,2 * E_{[24]} X^T$$

$$= 0,2 * \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} * \begin{bmatrix} -1 & -1 & +1 & -1 \\ +1 & -1 & -1 & +1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Et voilà !

Memória auto-associativa com aprendizagem de Widrow-Hoff

Iteração 24

- Etapa 4: Corrigir a matriz de pesos

$$W_{[24]} = W_{[23]} + \Delta w_{[24]}$$

$$= \begin{bmatrix} +0,333 & 0 & -0,333 & +0,333 \\ 0 & 1 & 0 & 0 \\ -0,333 & 0 & +0,333 & -0,333 \\ +0,333 & 0 & -0,333 & +0,333 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} +0,333 & 0 & -0,333 & +0,333 \\ 0 & 1 & 0 & 0 \\ -0,333 & 0 & +0,333 & -0,333 \\ +0,333 & 0 & -0,333 & +0,333 \end{bmatrix}$$

Memória auto-associativa com aprendizagem de Widrow-Hoff

Exercício 1

- Implementar em C++ o exemplo anterior.

Exercício 2

- O número de iterações necessário para se alcançar a solução correta depende da constante de aprendizagem escolhida.
- Se o valor da constante é muito grande, a solução oscila em torno da solução correta mas nunca chega nela. Se o valor da constante é muito pequeno, pode ser necessário um número infinito de iterações.
- Em quantas iterações a aprendizagem de Widrow-Hoff, para o exemplo trabalhado acima, converge para um erro nulo ao se adotar uma constante igual a 0,15?

Memória linear hetero-associativa

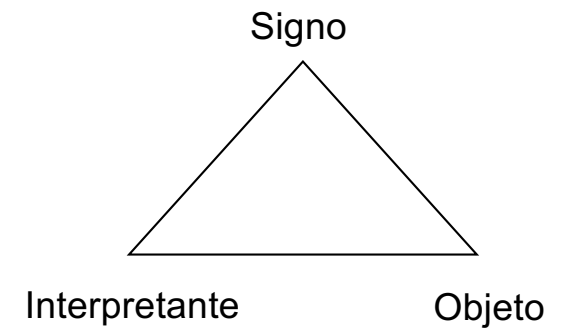
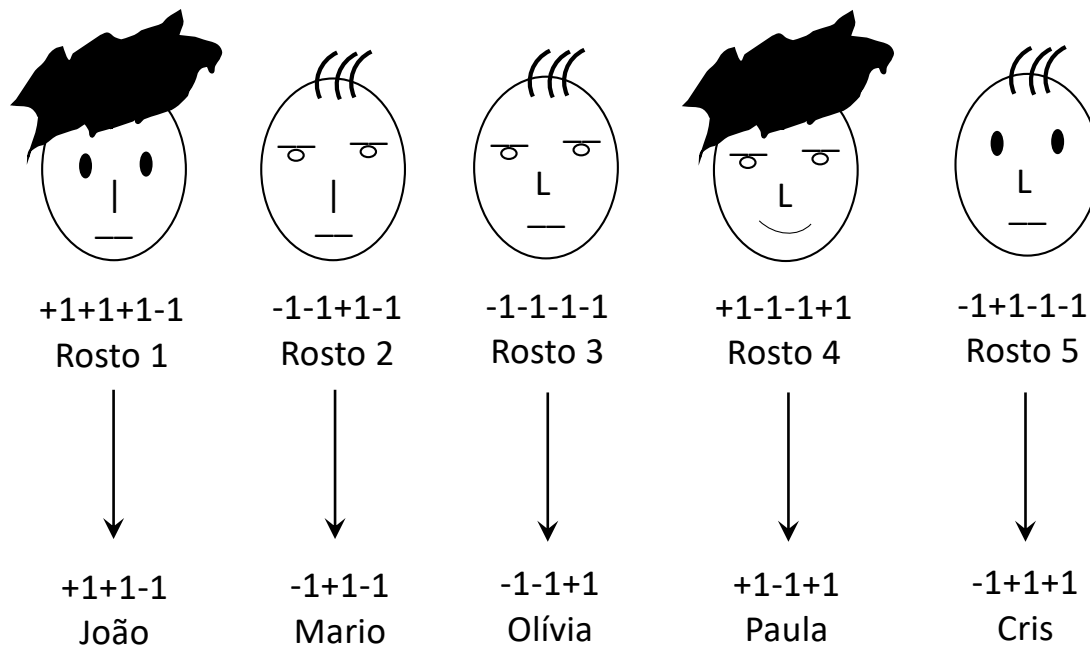
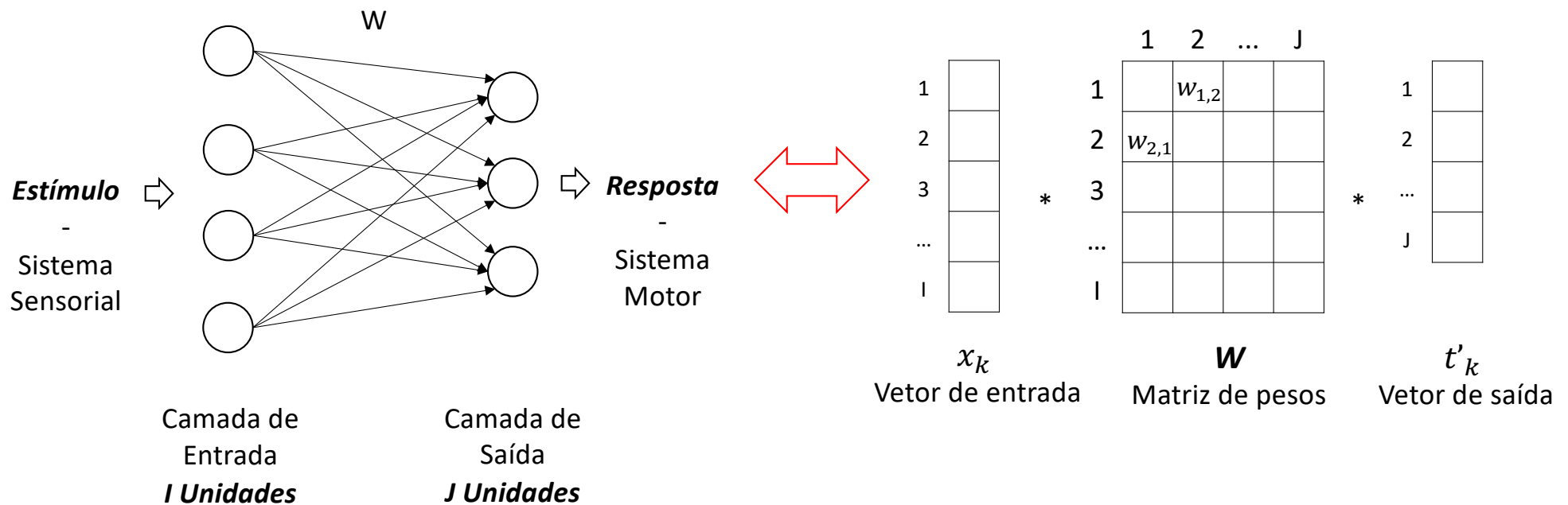


Figura: Semiótica peirceana

Memória linear hétero-associativa

- Como o estímulo apresentado na entrada é agora diferente do estímulo esperado na saída, a memória é dita hétero-associativa.



Memória linear hetero-associativa

- Cada rosto é representado por um vetor 4x1, notado x_k e o conjunto dos cinco rostos forma uma matriz 4x5:

$$X = \begin{bmatrix} 1 & -1 & -1 & 1 & -1 \\ 1 & -1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 & -1 \\ -1 & -1 & -1 & 1 & -1 \end{bmatrix}$$

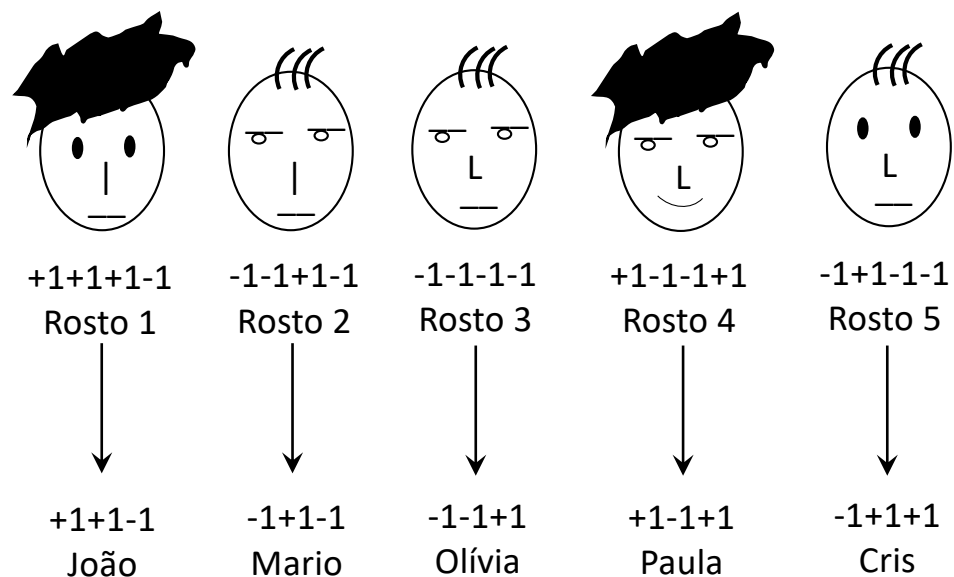
- Cada nome é representado por um vetor 3x1, notado t_k . O conjunto dos cinco nomes é representado por uma matriz 3x5:

$$T = \begin{bmatrix} 1 & -1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 & 1 \\ -1 & -1 & 1 & 1 & 1 \end{bmatrix}$$

Memória linear hetero-associativa

- O problema é ensinar a rede de neurônios a memorizar cada um dos pares entrada/saída que seguem:

- $x_1 \rightarrow t_1$
- $x_2 \rightarrow t_2$
- $x_3 \rightarrow t_3$
- $x_4 \rightarrow t_4$
- $x_5 \rightarrow t_5$



Memória linear hetero-associativa

A **regra de aprendizagem de Hebb** diz que a intensidade da conexão entre a i-ésima célula de entrada e a j-ésima de saída é proporcional à atividade das duas células.

- Se as duas células estão excitadas ou inibidas ao mesmo tempo, a intensidade da conexão aumenta.
- Se uma célula está excitada e a outra está inibida, e vice versa, a intensidade da conexão diminui.
- Se duas células não estão nem excitadas e nem inibidas, a intensidade da conexão entre elas fica inalterada.

$$W = \gamma \sum_{k=1}^k x_k t_k^T$$

γ representa uma constante de proporcionalidade ($\gamma=1$).

Memória linear hetero-associativa

- O primeiro rosto x_1 é associado ao seu nome t_1

$$\begin{aligned}w_{[1]} &= w_{[0]} + x_1 t_1^T \\&= \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} +1 \\ +1 \\ +1 \\ -1 \end{bmatrix} * \begin{bmatrix} +1 & +1 & -1 \end{bmatrix} \\&= \begin{bmatrix} +1 & +1 & -1 \\ +1 & +1 & -1 \\ +1 & +1 & -1 \\ -1 & -1 & +1 \end{bmatrix}\end{aligned}$$

Observa-se que a cada vez que uma célula de entrada e uma célula de saída são todas as duas positivas ou todas as duas negativas o peso entre elas aumenta em +1. A cada vez que duas células possuem valores opostos (+ e -, ou, - e +), o peso entre as duas células diminui de -1.

Memória linear hetero-associativa

- O segundo rosto x_2 é associado ao seu nome t_2

$$w_{[2]} = w_{[1]} + x_2 t_2^T$$

$$= \begin{bmatrix} +1 & +1 & -1 \\ +1 & +1 & -1 \\ +1 & +1 & -1 \\ -1 & -1 & +1 \end{bmatrix} + \begin{bmatrix} -1 \\ -1 \\ +1 \\ -1 \end{bmatrix} * [-1 \quad +1 \quad -1]$$

$$= \begin{bmatrix} +2 & 0 & 0 \\ +2 & 0 & 0 \\ 0 & +2 & -2 \\ 0 & -2 & +2 \end{bmatrix}$$

Memória linear hetero-associativa

- Assim sucessivamente. O quinto rosto x_5 é associado ao seu nome t_5

$$w_{[5]} = w_{[4]} + x_5 t_5^T$$

$$= \begin{bmatrix} +4 & 0 & 0 \\ +2 & +2 & -2 \\ 0 & +4 & -4 \\ +2 & -2 & +2 \end{bmatrix} + \begin{bmatrix} -1 \\ +1 \\ -1 \\ -1 \end{bmatrix} * [-1 \quad +1 \quad +1]$$

$$= \begin{bmatrix} +5 & -1 & -1 \\ +1 & +3 & -1 \\ +1 & +3 & -5 \\ +3 & -3 & +1 \end{bmatrix}$$

Memória linear hetero-associativa

- Como para a memória auto-associativa, a matriz de pesos pode ser obtida diretamente.

$$W = XT^T$$

$$= \begin{bmatrix} 1 & -1 & -1 & 1 & -1 \\ 1 & -1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 & -1 \\ -1 & -1 & -1 & 1 & -1 \end{bmatrix} * \begin{bmatrix} +1 & +1 & -1 \\ -1 & +1 & -1 \\ -1 & -1 & +1 \\ +1 & -1 & +1 \\ -1 & +1 & +1 \end{bmatrix}$$

$$= \begin{bmatrix} +5 & -1 & -1 \\ +1 & +3 & -1 \\ +1 & +3 & -5 \\ +3 & -3 & +1 \end{bmatrix}$$

Memória linear hétero-associativa

- A **lembrança** da memória hétero-associativa se obtém por:

$$t'_k = W^T x_k$$

- Como anteriormente, a qualidade desta resposta pode ser avaliada pelo cálculo do cosseno entre o vetor esperado, t_k e a resposta da memória, t'_k .

$$\cos(t'_k, t_k) = \frac{t'^T_k t_k}{\|t'_k\| \|t_k\|}$$

Memória linear hetero-associativa

- Se lembrando do nome de João (+1+1-1)

$$t'_1 = W^T x_1$$

$$= \begin{bmatrix} +5 & +1 & +1 & +3 \\ -1 & +3 & +3 & -3 \\ -1 & -1 & -5 & +1 \end{bmatrix} * \begin{bmatrix} +1 \\ +1 \\ +1 \\ -1 \end{bmatrix} = \begin{bmatrix} +4 \\ +8 \\ -8 \end{bmatrix}$$

- Para avaliar a resposta t'_1 , fazemos o $\cos(t'_1, t_1)$

Memória linear hetero-associativa

- Para avaliar a resposta t'_1 , fazemos o $\cos(t'_1, t_1)$

$$t'_1 = \begin{bmatrix} +4 \\ +8 \\ -8 \end{bmatrix} \text{ e } t_1 = \begin{bmatrix} +1 \\ +1 \\ -1 \end{bmatrix}$$

$$\begin{aligned} \cos(t'_1, t_1) &= \frac{t'^T_1 t_1}{\|t'_1\| \|t_1\|} \\ &= \frac{16}{\sqrt{96}\sqrt{3}} = \frac{16}{16,97} = 0,9428 \end{aligned}$$

Memória linear hetero-associativa

- As respostas para cada um dos cinco rostos aprendidos podem ser obtidos multiplicando a matriz dos rostos X pela transposta da matriz de pesos W .

$$T' = W^T X$$

$$\begin{aligned} &= \begin{bmatrix} +5 & +1 & +1 & +3 \\ -1 & +3 & +3 & -3 \\ -1 & -1 & -5 & +1 \end{bmatrix} * \begin{bmatrix} 1 & -1 & -1 & 1 & -1 \\ 1 & -1 & -1 & -1 & 1 \\ 1 & 1 & -1 & -1 & -1 \\ -1 & -1 & -1 & 1 & -1 \end{bmatrix} \\ &= \begin{bmatrix} +4 & -8 & -10 & +6 & -8 \\ +8 & +4 & -2 & -10 & +4 \\ -8 & -4 & +6 & +6 & +4 \end{bmatrix} \end{aligned}$$

ADALINE (*Adaptive Linear Neuron*)

- ADALINE é considerada uma das primeiras redes de neurônios artificiais. Proposta por Widrow & Hoff (1960), se utiliza da regra de aprendizagem Widrow-Hoff.
 - ADALINE é uma prima do perceptron, proposto por Rosenblatt em 1957 para modelar a atividade perceptiva humana.
 - As unidades de base do perceptron são neurônios de McCulloch & Pitts (1943).

McCulloch, Warren;
Pitts, Walter. **A Logical
Calculus of Ideas
Immanent in Nervous
Activity**. Bulletin of
Mathematical
Biophysics, 1943, 5,
115–133.

Warren S. McCulloch
(1898-1969) foi
professor de
psiquiatria, trabalhando
na área de
neurofisiologia e
cibernética. **Walter H.
Pitts** (1923-1969) foi
autodidata na área de
lógica e matemática.

ADALINE (*Adaptive Linear Neuron*)

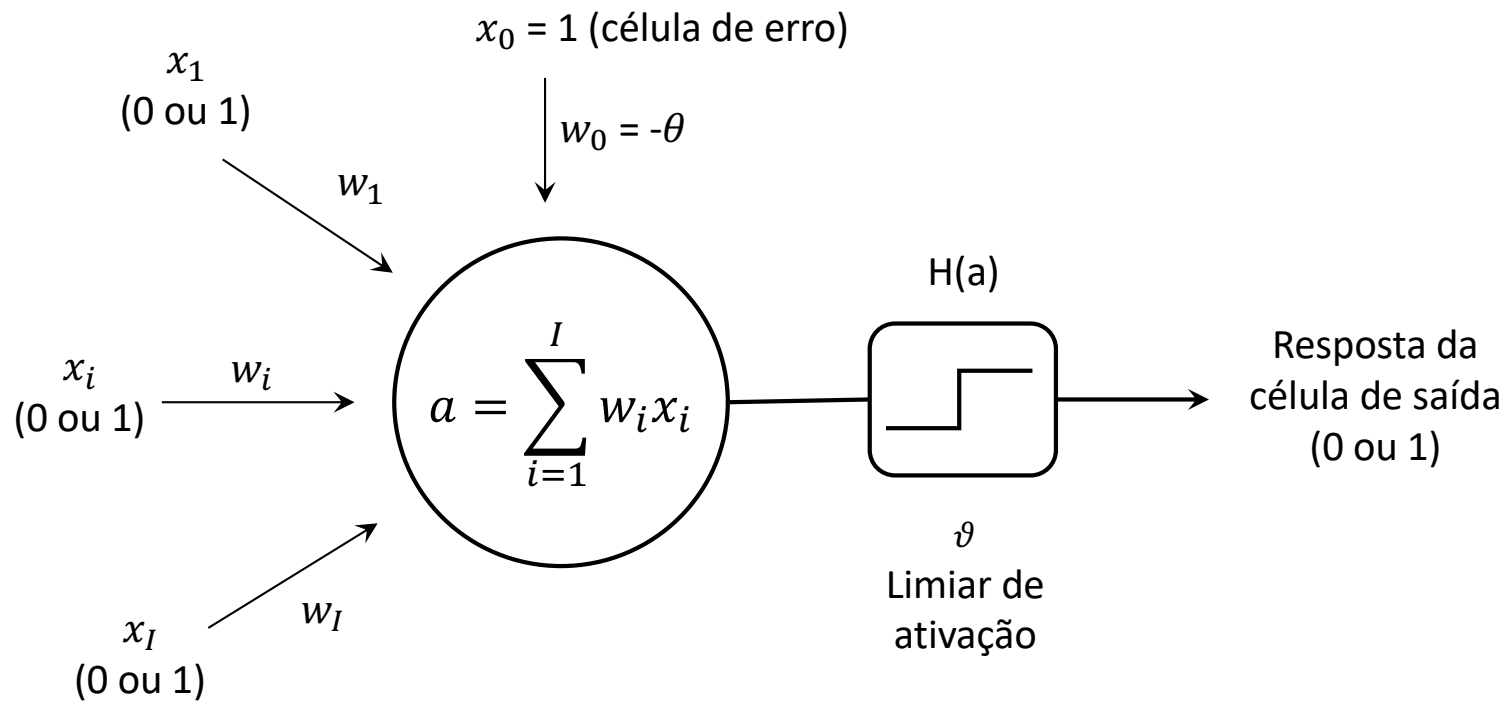


Figura: Um neurônio de McCulloch & Pitts calcula sua ativação a como a soma ponderada de suas entradas. A resposta é igual a 0, se $a \leq \vartheta$ e igual a 1, se $a > \vartheta$.

ADALINE (*Adaptive Linear Neuron*)

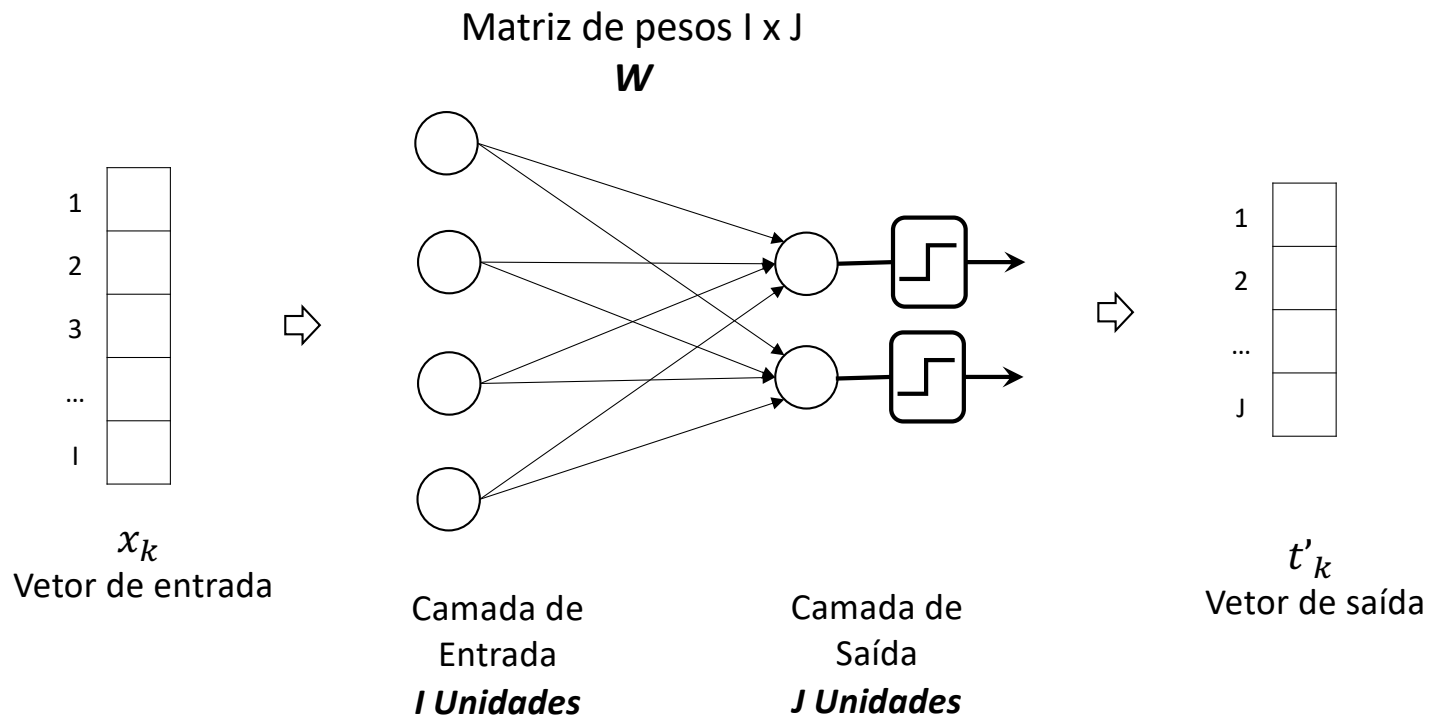


Figura: Um perceptron com I neurônios de entrada e J neurônios de saída.

ADALINE (*Adaptive Linear Neuron*)

- Formalmente, a atividade a de um neurônio de McCulloch & Pitts conectado a I neurônios de entrada é obtida por:

$$a = \sum_{i=1}^I w_i x_i = w_1 x_1 + w_2 x_2 + \dots + w_i x_i + \dots + w_I x_I$$

x_i representa o estado de ativação dos neurônios de entrada

w_i representa a intensidade da conexão entre o i -ésimo neurônio de entrada e o neurônio de McCulloch & Pitts

- Resposta, $h(a)$

$$h(a) = \begin{cases} 0, & \text{para } a \leq \vartheta \\ 1, & \text{para } a > \vartheta \end{cases}$$

ADALINE (*Adaptive Linear Neuron*)

ADALINE

- Para cada vetor de entrada x_k (composto de l elementos), associamos a resposta de ADALINE a_k obtida multiplicando a ativação das células de entrada pelo vetor de conexões w .
- A resposta teórica (binária) é t_k , a resposta efetiva é $o_k = a_k = w^T x_k$
- O erro para a resposta i é a diferença entre a resposta teórica e a resposta efetiva.

$$e_k = t_k - o_k = t_k - a_k = t_k - w^T x_k$$

ADALINE (*Adaptive Linear Neuron*)

- Opta-se pelo uso do quadrado do erro como medida de performance, visando usar recursos do **cálculo diferencial**. O erro do quadrado se calcula assim:

$$e_k = t_k - o_k = t_k - a_k = t_k - w^T x_k$$

$$e_k^2 = (t_k - a_k)^2 = (t_k^2 + a_k^2 - 2a_k t_k) = t_k^2 + x_k^T w w^T x_k - 2t_k w^T x_k$$

- O gradiente de e_k^2 se calcula assim:

$$\frac{\partial e_k^2}{\partial w} = \nabla_{e_k^2} = 2w^T x_k x_k^T - 2t_k x_k^T = -2(t_k - w^T x_k) x_k^T$$

ADALINE (*Adaptive Linear Neuron*)

- O método do gradiente consiste em diminuir os parâmetros de uma função no sentido inverso do gradiente desta função. O gradiente dá a direção do maior aumento, sua inversa dá a direção da maior diminuição.
- Como procuramos minimizar e_k^2 , modificamos os parâmetros do vetor w no sentido inverso do gradiente. Assim Δw na apresentação do estímulo k é:

$$\Delta w = -\eta \nabla_{e_k^2} = \eta (t_k - w^T x_k) x_k^T$$

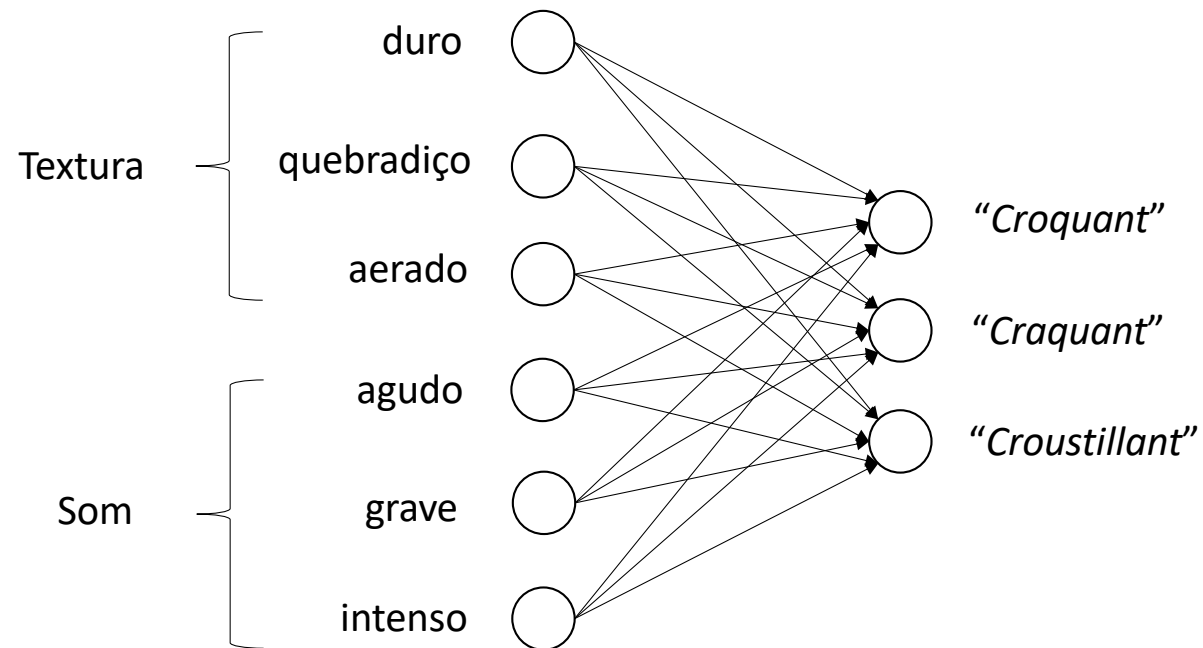
ADALINE (*Adaptive Linear Neuron*)

- Para uma aprendizagem por pacote, a regra delta para ADALINE pode ser escrita da seguinte forma:

$$\Delta W = \eta (EX^T)^T = \eta XE^T$$

E sendo a matriz de erro: $E = T - O$

ADALINE (*Adaptive Linear Neuron*)



ADALINE (*Adaptive Linear Neuron*)

Tabela: um exemplo de 9 produtos alimentares descritos por suas características de textura e som à mordida

	textura			som			resposta		
	duro	quebradiço	aerado	agudo	grave	intenso	"croustillant"	"craquant"	"croquant"
chocolate	1	0	0	0	1	1	0	1	0
rabanete	1	0	0	0	1	0	0	1	0
maça	0	0	0	0	1	1	0	1	0
biscoito amantegado	0	1	0	1	0	0	0	0	1
biscoito spéculos	1	0	0	1	0	1	0	0	1
língua de gato	1	1	0	1	0	1	0	0	1
biscoito craquette	0	1	1	1	0	1	1	0	0
salgadinho curly	0	1	1	0	1	0	1	0	0
Folheado	0	1	1	1	0	0	1	0	0

ADALINE (*Adaptive Linear Neuron*)

- Formalmente o problema é o de associar cada vetor de entrada x_k ao vetor de saída correspondente a t_k .

$$X = \begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

O conjunto de alimentos fica representado pela matriz X de 6x9

$$T = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}$$

As categorias “croustillant”, “craquant” ou “croquant” de cada alimento ficam representadas na matriz T de 3x9.

ADALINE (*Adaptive Linear Neuron*)

- Ao invés de inicializarmos a matriz de pesos W com valores nulos, vamos inicializa-la com valores aleatórios entre -0,6 e 0,6.

$$W = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \Rightarrow W_{[0]} = \begin{bmatrix} -0,4410 & +0,0369 & +0,1774 \\ -0,1722 & -0,0568 & -0,3871 \\ +0,4373 & +0,0138 & +0,5009 \\ -0,2144 & +0,3218 & -0,1554 \\ +0,0561 & +0,4341 & -0,1156 \\ -0,1280 & +0,3057 & -0,3583 \end{bmatrix}$$

ADALINE (*Adaptive Linear Neuron*)

- A matriz de resposta é calculada por:

$$O = W^T X$$

- Cálculo da matriz de erro E:

$$E = T - O$$

- Obtemos assim um valor de erro igual a e .
- Para corrigir a matriz de pesos W , adotamos $\eta = 0,01$.

$$\Delta W = \eta (EX^T)^T = \eta X E^T$$

$$W_{[n+1]} = W_{[n]} + \Delta w_{[n]}$$

$$W = \gamma \sum_{k=1}^k x_k t_k^T$$

Aprendizagem
hetero-associativa

Aprendizagem
auto-associativa

$$\begin{aligned} W_{[n+1]} &= W_{[n]} + \eta (X - W_{[n]}X)X^T \\ &= W_{[n]} + \eta (X - X')X^T \\ &= W_{[n]} + \eta EX^T \\ &= W_{[n]} + \Delta w_{[n]} \end{aligned}$$

ADALINE (*Adaptive Linear Neuron*)

- Calcula-se a matriz de resposta.

$$O = W^T X$$

$$T = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} -0,5129 & -0,3849 & -0,0719 & -0,3866 & -0,7833 & -0,9556 & -0,0773 & +0,3211 & +0,0507 \\ +0,7767 & +0,4710 & +0,7398 & +0,2651 & +0,6644 & +0,6077 & +0,5846 & +0,3911 & +0,2789 \\ -0,2966 & +0,0617 & -0,4740 & -0,5425 & -0,3364 & -0,7235 & -0,3999 & -0,0018 & -0,0416 \end{bmatrix}$$

- Calcula-se a matriz de erro.

$$E = T - O$$

- Obtemos assim um valor de erro igual a $e = 15,4830$.

ADALINE (*Adaptive Linear Neuron*)

- Iteração 1.

$$W_{[1]} = \begin{bmatrix} -0,4156 & +0,0318 & +0,2090 \\ -0,1339 & -0,0772 & -0,3528 \\ +0,4628 & +0,0019 & +0,5034 \\ -0,1747 & +0,2986 & -0,1075 \\ +0,0719 & +0,4403 & -0,1093 \\ -0,0959 & +0,2925 & -0,3185 \end{bmatrix}$$

ADALINE (*Adaptive Linear Neuron*)

- Calcula-se novamente a matriz de resposta.

$$O = W^T X$$

$$= \begin{bmatrix} -0,4397 & -0,3437 & -0,0241 & -0,3086 & -0,6862 & -0,8201 & -0,0583 & +0,4008 & +0,1542 \\ +0,7646 & +0,4721 & +0,7328 & +0,2213 & +0,6228 & +0,5456 & +0,5157 & +0,3650 & +0,2232 \\ -0,2188 & +0,0997 & -0,4278 & -0,4603 & -0,2170 & -0,5698 & -0,2754 & +0,0414 & +0,0431 \end{bmatrix}$$

- Calcula-se a matriz de erro.

$$E = T - O$$

- Obtemos assim um valor de erro igual a $e = 13,8868$.

ADALINE (*Adaptive Linear Neuron*)

- Iteração 2.

$$W_{[2]} = \begin{bmatrix} -0,3935 & +0,0278 & +0,2369 \\ -0,1005 & -0,0952 & -0,3225 \\ +0,4854 & +0,0086 & +0,5038 \\ -0,1406 & +0,2781 & -0,0650 \\ +0,0852 & +0,4470 & -0,1052 \\ -0,0684 & +0,2813 & -0,2835 \end{bmatrix}$$

ADALINE (*Adaptive Linear Neuron*)

- Calcula-se novamente a matriz de resposta.

$$O = W^T X$$

$$= \begin{bmatrix} -0,3767 & -0,3083 & +0,0168 & -0,2411 & -0,6024 & -0,7029 & +0,1759 & +0,4700 & +0,2443 \\ +0,7561 & +0,4748 & +0,7283 & +0,1829 & +0,5872 & +0,4920 & +0,4556 & +0,3432 & +0,1743 \\ -0,1518 & +0,1317 & -0,3887 & -0,3875 & -0,1116 & -0,4341 & -0,1672 & +0,0761 & +0,1163 \end{bmatrix}$$

- Calcula-se a matriz de erro.

$$E = T - O$$

- Obtemos assim um valor de erro igual a $e = 12,5989$.

ADALINE (*Adaptive Linear Neuron*)

- Após mais de 300 iterações.

$$O^T = \begin{bmatrix} -0,0214 & +0,9229 & +0,1324 \\ +0,0102 & +0,7644 & +0,1901 \\ +0,0737 & +0,8214 & +0,0128 \\ +0,1024 & -0,1421 & +0,9240 \\ -0,0029 & +0,2625 & +0,7035 \\ -0,0244 & +0,1178 & +0,9859 \\ +0,9642 & -0,0925 & +0,1676 \\ +0,9773 & +0,4094 & -0,3457 \\ +0,9958 & -0,2510 & +0,2253 \end{bmatrix}$$

$$T^T = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

Com $e = 3,5251$ a resposta não é perfeita, mas ela constitui uma boa estimativa da matriz inicial.

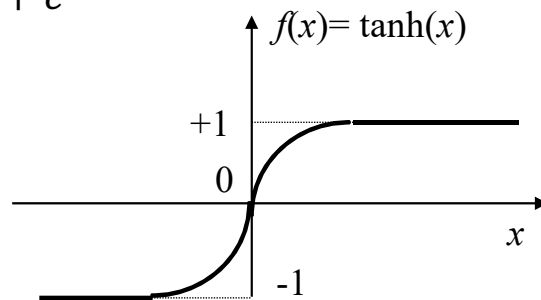
ADALINE logística

A performance do ADALINE clássico pode ser melhorada usando modelos não-lineares. Vejamos o ADALINE sigmoide.

- Utiliza uma função de ativação diferencial, sigmoide, $f(x)$:

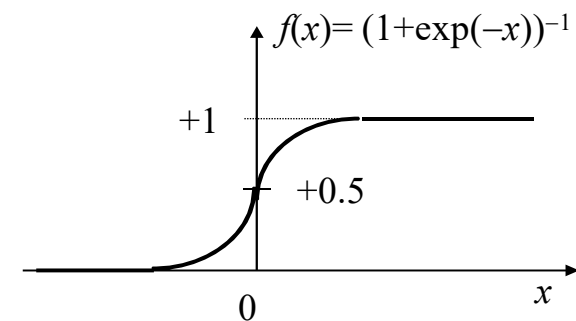
A tangente hiperbólica:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



A função logística:

$$f(x) = \frac{1}{1 + e^{-x}}$$



ADALINE logística

- Regra de aprendizagem de Widrow-Hoff para uma ADALINE logística

$$o = f(a) = \frac{1}{1 + e^{-a}}$$

O **número e** é uma constante matemática que é a base dos logaritmos naturais. É chamado de número de Euler, número de Napier, número neperiano e outros. O valor aproximado é 2,718281828459045235360287.

ADALINE logística

- Regra de aprendizagem de Widrow-Hoff para uma ADALINE logística

$$o_k = f(w^T x_k)$$

Com:

$$e_k = t_k - o_k = t_k - f(a_k) = t_k - f(w^T x_k)$$

$$e_k^2 = (t_k - o_k)^2 = (t_k^2 + o_k^2 - 2o_k t_k)$$

$$o = f(a) = \frac{1}{1 + e^{-a}}$$

Com:

$$f'(x) = \frac{df}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = f(x)[1 - f(x)]$$

ADALINE logística

- O gradiente de e_k^2 se calcula assim:

$$\begin{aligned}\frac{\partial e_k^2}{\partial w} &= \nabla_{e_k^2} \\ &= \frac{\partial e_k^2}{\partial (w^T x)} \frac{\partial f(w^T x)}{\partial w^T x} \frac{\partial w^T x}{\partial w} \\ &= -2[t_k - f(w^T x)]f'(w^T x)x^T \\ &= -2(t_k - o_k)f'(a_k) \\ &= -2(t_k - o_k)o_k(1 - o_k)x^T\end{aligned}$$

ADALINE
clássica


$$\begin{aligned}\frac{\partial e_k^2}{\partial w} &= \nabla_{e_k^2} = 2w^T x_k x_k^T - 2t_k x_k^T \\ &= -2(t_k - w^T x_k)x_k^T\end{aligned}$$

ADALINE logística

- Como procuramos *minimizar* e_k^2 , modificamos os parâmetros do vetor w no sentido inverso do gradiente. Assim Δw na apresentação do estímulo k é:

$$\Delta w = -\eta \nabla_{e_k^2} = \eta(t_k - o_k)o_k(1 - o_k)x_k^T$$

- Para uma aprendizagem por pacote temos:

$$\begin{aligned}\Delta W &= \eta\{[E \odot O \odot (1 - O)] * X^T\}^T \\ &= \eta X * \{[E \odot O \odot (1 - O)]^T\}\end{aligned}$$

ADALINE
clássica



$$\Delta w = -\eta \nabla_{e_k^2} = \eta(t_k - w^T x_k)x_k^T$$

$$\Delta W = \eta(EX^T)^T = \eta XE^T$$

ADALINE logística

- Apliquemos ADALINE logística ao problema de categorização de alimentos.

$$W_{[0]} = \begin{bmatrix} -0,4410 & +0,0369 & +0,1774 \\ -0,1722 & -0,0568 & -0,3871 \\ +0,4373 & +0,0138 & +0,5009 \\ -0,2144 & +0,3218 & -0,1554 \\ +0,0561 & +0,4341 & -0,1156 \\ -0,1280 & +0,3057 & -0,3583 \end{bmatrix}$$

$$X = \begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix}$$

Matriz de pesos inicializada com
valores aleatórios

ADALINE logística

$$O = f\{W^T X\}$$

$$= f\left\{\begin{bmatrix} -0,5129 & -0,3849 & -0,0719 & -0,3866 & -0,7833 & -0,9556 & -0,0773 & +0,3211 & +0,0507 \\ +0,7767 & +0,4710 & +0,7398 & +0,2651 & +0,6644 & +0,6077 & +0,5846 & +0,3911 & +0,2789 \\ -0,2966 & +0,0617 & -0,4740 & -0,5425 & -0,3364 & -0,7235 & -0,3999 & -0,0018 & -0,0416 \end{bmatrix}\right\}$$

$$= \begin{bmatrix} \frac{1}{1+\exp(+0,5129)} & \cdots & \frac{1}{1+\exp(-0,0507)} \\ \vdots & \ddots & \vdots \\ \frac{1}{1+\exp(+0,2966)} & \cdots & \frac{1}{1+\exp(+0,0416)} \end{bmatrix}$$

$$= \begin{bmatrix} 0,3745 & 0,4049 & 0,4820 & 0,4045 & 0,3136 & 0,2778 & 0,4807 & 0,5796 & 0,5127 \\ 0,6850 & 0,6156 & 0,6770 & 0,5659 & 0,6602 & 0,6474 & 0,6421 & 0,5965 & 0,5693 \\ 0,4264 & 0,5154 & 0,3837 & 0,3676 & 0,4167 & 0,3266 & 0,4013 & 0,4996 & 0,4896 \end{bmatrix}$$

ADALINE logística

- A matriz de erro: $E = T - O$

$$T = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$O = \begin{bmatrix} 0,3745 & 0,4049 & 0,4820 & 0,4045 & 0,3136 & 0,2778 & 0,4807 & 0,5796 & 0,5127 \\ 0,6850 & 0,6156 & 0,6770 & 0,5659 & 0,6602 & 0,6474 & 0,6421 & 0,5965 & 0,5693 \\ 0,4264 & 0,5154 & 0,3837 & 0,3676 & 0,4167 & 0,3266 & 0,4013 & 0,4996 & 0,4896 \end{bmatrix}$$

$$E = \begin{bmatrix} -0,3745 & -0,4049 & -0,4820 & -0,4045 & -0,3136 & -0,2778 & +0,5193 & +0,4204 & +0,4873 \\ +0,3150 & +0,3844 & +0,3230 & -0,5659 & -0,6602 & -0,6474 & -0,6421 & -0,5965 & -0,5693 \\ -0,4264 & -0,5154 & -0,3837 & +0,6324 & +0,5833 & +0,6734 & -0,4013 & -0,4996 & -0,4896 \end{bmatrix}$$

ADALINE logística

- A matriz de correção: $\Delta W = \eta X * \{[E \odot O \odot (1 - O)]^T\}$ $\eta = 0,01$

$$= 0,01 * \begin{bmatrix} 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \end{bmatrix} *$$

$$\left\{ \begin{bmatrix} -0,375 & +0,315 & -0,426 \\ -0,405 & +0,384 & -0,515 \\ -0,482 & +0,323 & -0,384 \\ -0,405 & -0,566 & +0,632 \\ -0,314 & -0,660 & +0,583 \\ -0,278 & -0,647 & +0,673 \\ +0,519 & -0,642 & -0,401 \\ +0,420 & -0,597 & -0,500 \\ +0,487 & -0,569 & -0,490 \end{bmatrix} \odot \begin{bmatrix} 0,375 & 0,685 & 0,426 \\ 0,405 & 0,616 & 0,515 \\ 0,482 & 0,677 & 0,384 \\ 0,405 & 0,566 & 0,368 \\ 0,314 & 0,660 & 0,417 \\ 0,278 & 0,647 & 0,327 \\ 0,481 & 0,642 & 0,401 \\ 0,580 & 0,597 & 0,500 \\ 0,513 & 0,569 & 0,490 \end{bmatrix} \odot \begin{bmatrix} 0,625 & 0,315 & 0,574 \\ 0,595 & 0,384 & 0,485 \\ 0,518 & 0,323 & 0,616 \\ 0,596 & 0,434 & 0,632 \\ 0,686 & 0,340 & 0,583 \\ 0,722 & 0,353 & 0,673 \\ 0,519 & 0,358 & 0,599 \\ 0,420 & 0,403 & 0,500 \\ 0,487 & 0,431 & 0,510 \end{bmatrix} \right\}$$

ADALINE logística

- A matriz de correção: $\Delta W = \eta X * \{[E \odot O \odot (1 - O)]^T\}$ $\eta = 0,01$

$$= 0,01 * \begin{bmatrix} -0,3085 & -0,1370 & +0,0568 \\ +0,2007 & -0,7175 & -0,0485 \\ +0,3538 & -0,4307 & -0,3437 \\ +0,0307 & -0,7220 & +0,2181 \\ -0,2032 & +0,0860 & -0,4487 \\ -0,2017 & -0,3048 & -0,0016 \end{bmatrix}$$

ADALINE logística

- Após 30 mil iterações a matriz de resposta está bem próxima da matriz esperada:

$$O^T = \begin{bmatrix} 0,0019 & 0,9911 & 0,0080 \\ 0,0037 & 0,9888 & 0,0108 \\ 0,0151 & 0,9888 & 0,0028 \\ 0,0204 & 0,0001 & 0,9825 \\ 0,0023 & 0,0140 & 0,9887 \\ 0,0014 & 0,0001 & 0,9917 \\ 0,9851 & 0,0000 & 0,0112 \\ 0,9907 & 0,0130 & 0,0000 \\ 0,9922 & 0,0000 & 0,0150 \end{bmatrix}$$

$$T^T = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

O erro é igual a: **$e = 0,2201$** . Este erro é claramente inferior ao obtido com ADALINE clássico ($e = 3,5251$) e exemplifica a vantagem dos modelos não-lineares.

ADALINE

Exercício

- Para o problema de associação de nomes aos rostos apresentado na exposição da memória linear hetero-associativa, usar ADALINE logística.

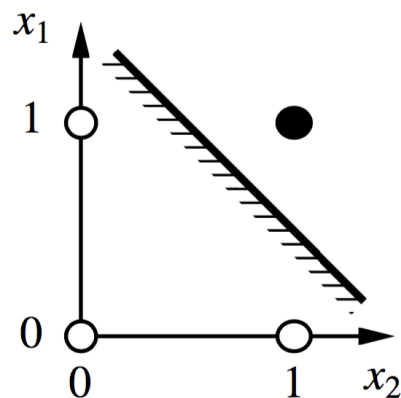
Perceptron, Funções Lógicas e o problema XOR

Tabela: Conectivos lógicos

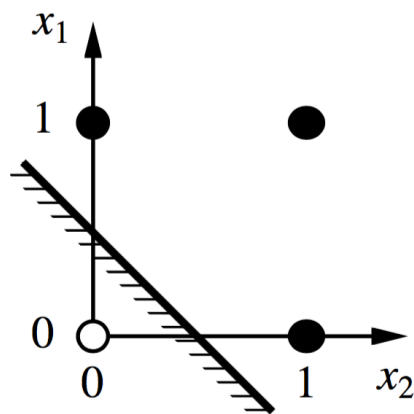
a	b	a OU b	a E b	NÃO a	a XOU b
1	1	1	1	0	0
1	0	1	0	0	1
0	1	1	0	1	1
0	0	0	0	1	0

Se a e b são representados pelos neurônios formais α e β , o valor da direita é resultado das entradas e do circuito de ativações.

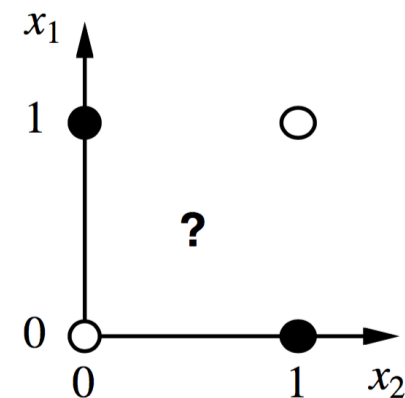
Perceptron, Funções Lógicas e o problema XOR



(a) x_1 **and** x_2



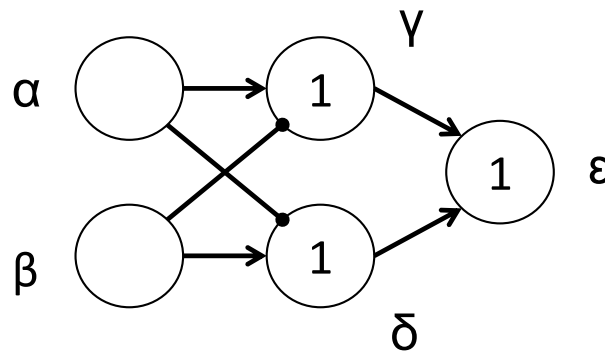
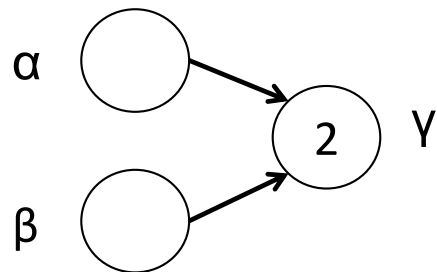
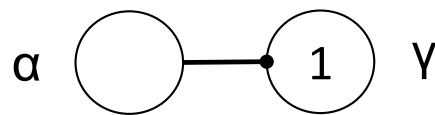
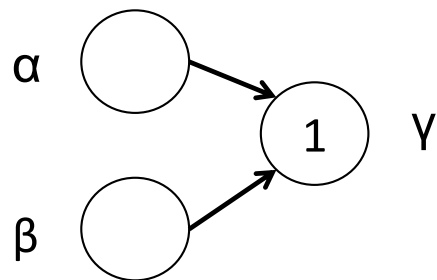
(b) x_1 **or** x_2



(c) x_1 **xor** x_2

Figura: “Os pontos pretos indicam um ponto no espaço de entrada onde o valor da função é 1, e pontos brancos indicam um ponto onde o valor é 0. O perceptron retorna 1 na região do lado não sombreado da linha. Em (c), não existe essa linha que classifica corretamente as entradas.” (Russell & Norvig, 1995)

Perceptron, Funções Lógicas e o problema XOR

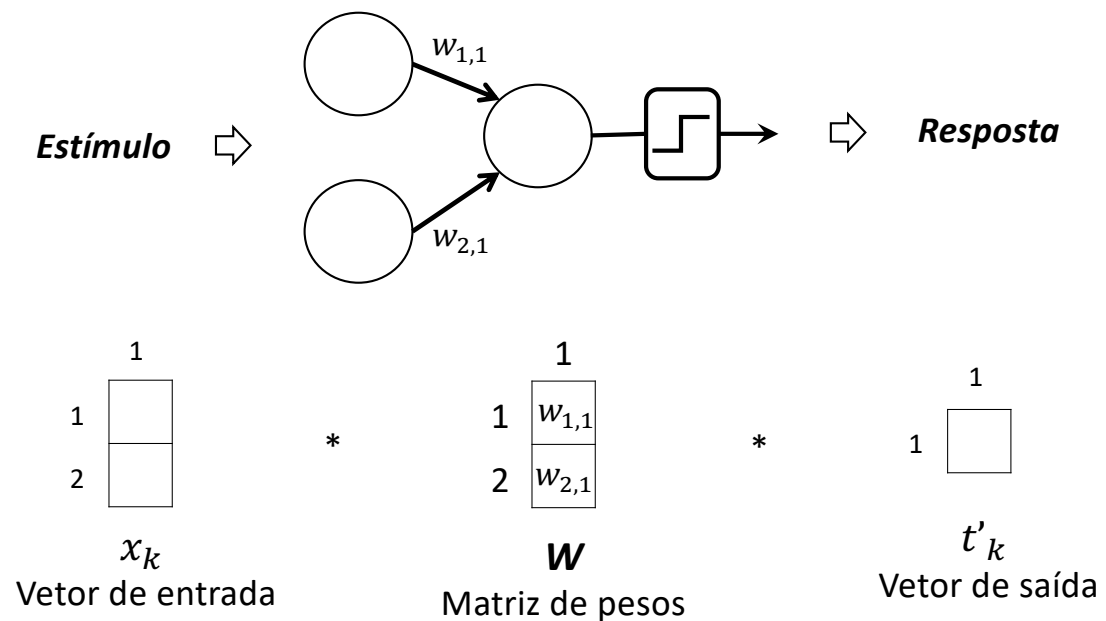


PULVERMÜLLER, F.
*The Neuroscience of
Language: On Brain
Circuits of Words and
Serial Order.* New York:
Cambridge University
Press, 2002.

Perceptron, Funções Lógicas e o problema XOR

- Apliquemos ADALINE logística para a aprendizagem da função OU

a	b	a OU b
1	1	1
1	-1	1
-1	1	1
-1	-1	-1



Perceptron, Funções Lógicas e o problema XOR

Inicialização

$$W_{[0]} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$X = \begin{bmatrix} +1 & +1 & -1 & -1 \\ +1 & -1 & +1 & -1 \end{bmatrix}$$

$$T = [+1 \quad +1 \quad +1 \quad -1]$$

Perceptron, Funções Lógicas e o problema XOR

Iteração 1

$$O = f\{W^T X\}$$

$$W^T X = [0 \quad 0] * \begin{bmatrix} +1 & +1 & -1 & -1 \\ +1 & -1 & +1 & -1 \end{bmatrix}$$

$$= [0 \quad 0 \quad 0 \quad 0]$$

$$O = [0,5 \quad 0,5 \quad 0,5 \quad 0,5]$$

$$E = T - O$$

$$= [+0,5 \quad +0,5 \quad +0,5 \quad -1,5]$$

Perceptron, Funções Lógicas e o problema XOR

Iteração 1

$$\Delta W = \eta X * \{[E \odot O \odot (1 - O)]^T\} \quad \eta = 1$$

$$\begin{bmatrix} +0,5 \\ +0,5 \\ +0,5 \\ -1,5 \end{bmatrix} \odot \begin{bmatrix} +0,5 \\ +0,5 \\ +0,5 \\ +0,5 \end{bmatrix} \odot \begin{bmatrix} +0,5 \\ +0,5 \\ +0,5 \\ +0,5 \end{bmatrix} = \begin{bmatrix} +0,125 \\ +0,125 \\ +0,125 \\ -0,375 \end{bmatrix}$$

$$\Delta W = \begin{bmatrix} 0,5 \\ 0,5 \end{bmatrix}$$

Perceptron, Funções Lógicas e o problema XOR

Iteração 1

$$W_{[n+1]} = W_{[n]} + \Delta w_{[n]}$$

$$= \begin{bmatrix} 0,5 \\ 0,5 \end{bmatrix}$$

Perceptron, Funções Lógicas e o problema XOR

Iteração 2

$$O = f\{W^T X\}$$

$$W^T X = [0,5 \quad 0,5] * \begin{bmatrix} +1 & +1 & -1 & -1 \\ +1 & -1 & +1 & -1 \end{bmatrix}$$

$$= [1 \quad 0 \quad 0 \quad -1]$$

$$O = [0,7311 \quad 0,5 \quad 0,5 \quad 0,2689]$$

$$E = T - O$$

$$= [+0,2689 \quad +0,5 \quad +0,5 \quad -1,2689]$$

Perceptron, Funções Lógicas e o problema XOR

$$\Delta W = \eta X * \{[E \odot O \odot (1 - O)]^T\} \quad \eta = 1$$

Iteração 2

$$\begin{bmatrix} +0,2689 \\ +0,5 \\ +0,5 \\ -1,2689 \end{bmatrix} \odot \begin{bmatrix} +0,7311 \\ +0,5 \\ +0,5 \\ +0,2689 \end{bmatrix} \odot \begin{bmatrix} +0,2689 \\ +0,5 \\ +0,5 \\ +0,7311 \end{bmatrix} = \begin{bmatrix} +0,0529 \\ +0,125 \\ +0,125 \\ -0,2495 \end{bmatrix}$$

$$\Delta W = \begin{bmatrix} 0,3024 \\ 0,3024 \end{bmatrix}$$

Perceptron, Funções Lógicas e o problema XOR

Iteração 2

$$W_{[n+1]} = W_{[n]} + \Delta w_{[n]}$$

$$= \begin{bmatrix} 0,8027 \\ 0,8027 \end{bmatrix}$$

Repetir por mais de 20 iterações.

O que fazer para evitar problemas de “overflow”?

Perceptron, Funções Lógicas e o problema XOR

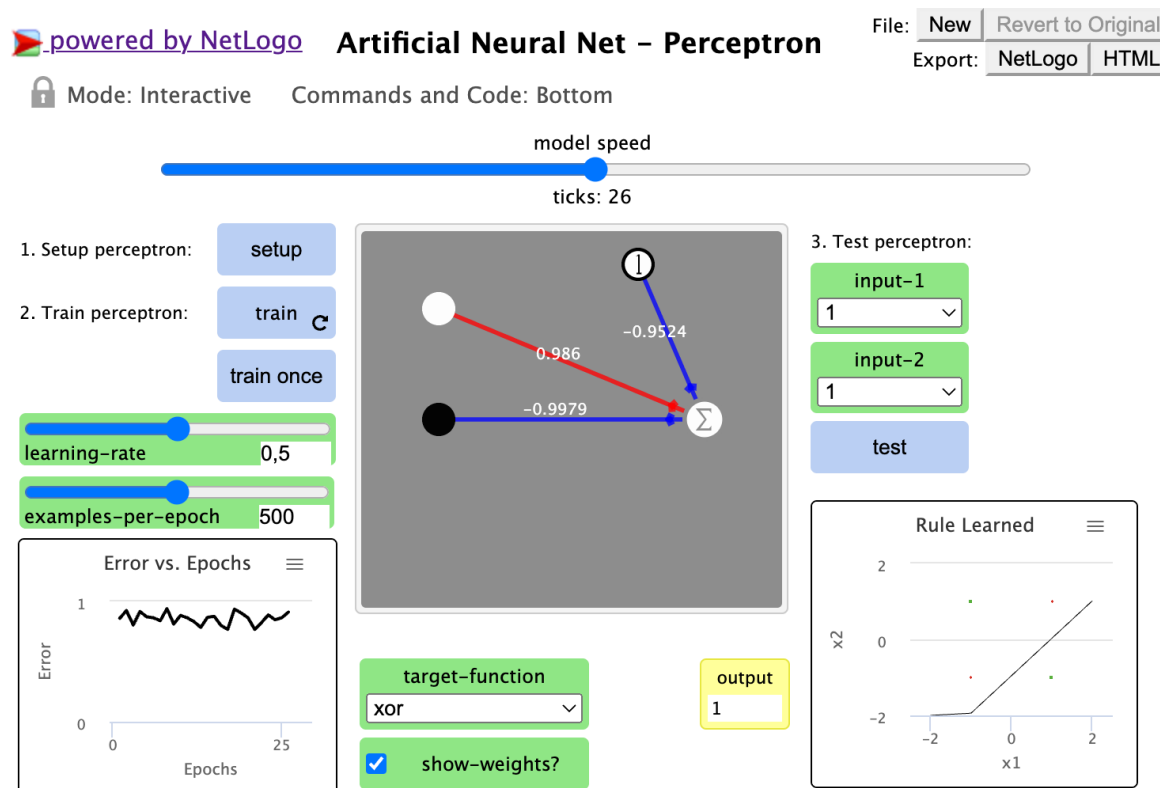


Figura: Uma rede neural artificial com uma célula de saída e duas de entrada, implementada no NetLogo® para aprendizagem de funções lógicas. Os dados apresentados exemplificam a incapacidade deste modelo de aprendizagem da função XOR.

Fonte: <http://www.netlogoweb.org/launch#http://ccl.northwestern.edu/netlogo/models/models>

Perceptron, Funções Lógicas e o problema XOR

Exercício

- Aplicar ADALINE logística para aprendizagem das funções lógicas E e XOR.
- Implementar o modelo em C++.
- Explicar a impossibilidade do perceptron proposto por Rosenblatt (1957) de aprender o conectivo XOR.

Perceptron Multicamadas & Funções Lógicas

- Perceptron multicamadas possui mais de uma camada de células podendo aprender.
- Se trata de uma rede com retropropagação do erro.
- Para aprender uma função binária (como exemplo as funções lógicas) é preciso visar não seus valores exatos (0 ou 1) mas valores aproximativos como (0,1 e 0,9), evitando um erro de “overflow”.
- Os valores 0 e 1 são assíntotas da função logística e assim só serão alcançadas quando a ativação das células de saída se tornam infinitas.

Perceptron Multicamadas & Funções Lógicas

- A rede aprende, de modo supervisionado, diminuindo o erro a cada iteração em função da resposta esperada. Isso é feito modificando a intensidade das conexões no sentido inverso do sinal de erro.
- Para um perceptron multicamadas, o ponto é aplicar essa mesma ideia também para as células intermediárias.
- O que precisamos observar é o modo de transmitir esse sinal de erro para as células intermediárias.
 - Cada célula da camada externa calcula seu sinal de erro.
 - Em seguida o sinal de erro é enviado às células da camada intermediária como uma média do erro, ponderada pela intensidade das conexões.

Perceptron Multicamadas & Funções Lógicas

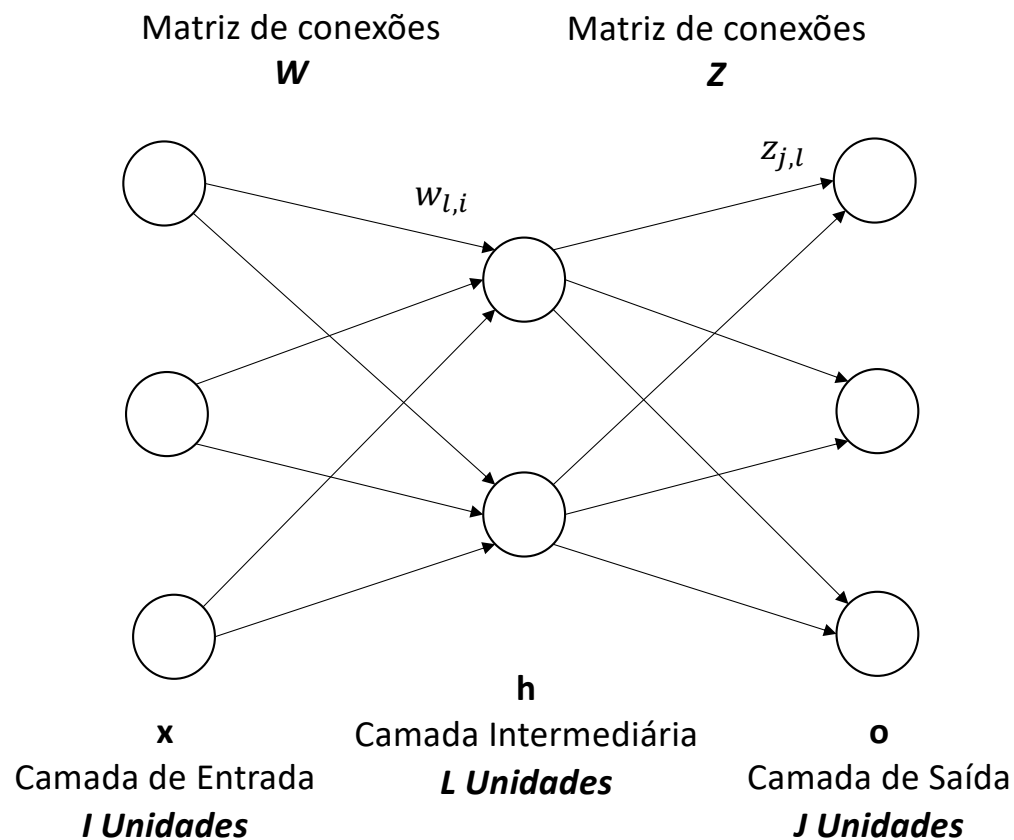


Figura: Um estímulo na entrada é notado x , a resposta na unidade intermediária a este estímulo é notada h (para *hidden*), a resposta da camada de saída é notada o e a resposta teórica ou esperada pela rede é notada t . As sinapses $w_{l,i}$ e $z_{j,l}$ são modificáveis por retropropagação do erro. A função que transforma a ativação em resposta deve ser não-linear (ex: uma função logística).

Perceptron Multicamadas & Funções Lógicas

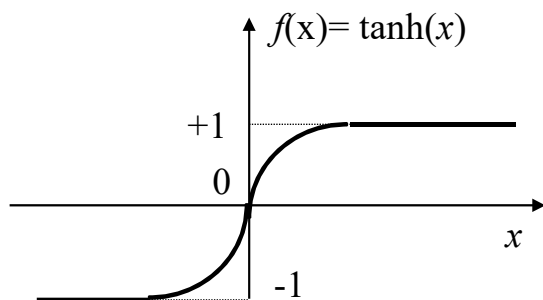
- x_k : vetor de I elementos representando o estímulo k . X é a matriz $I \times K$.
- h_k : vetor de L elementos representando a resposta das células intermediárias.
- o_k : vetor de J elementos representando a resposta das células de saída.
- t_k : vetor de J elementos representando a resposta desejada das células de saída para o estímulo k . T é a matriz $J \times K$ de respostas desejadas.
- W : matriz de pesos $L \times I$ das conexões entre as células da camada de entrada e as da camada intermediária ($w_{l,i}$).
- Z : matriz $J \times L$ dos valores de conexões entre as células da camada intermediária e as da camada de saída ($z_{j,l}$).

Perceptron Multicamadas & Funções Lógicas

- Notando a_n o estado de ativação de uma célula n , podendo ser da camada intermediária como da camada de saída, temos: $o_n = f(a_n)$

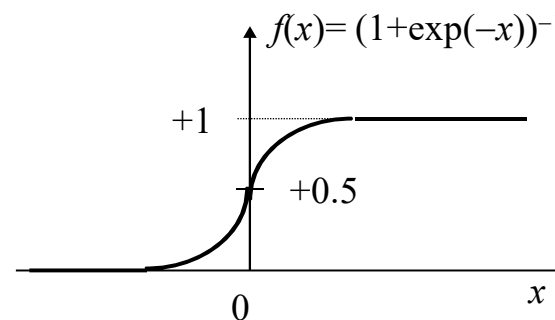
A tangente hiperbólica:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad \frac{d \tanh(x)}{dx} = [\operatorname{sech}(x)]^2 = \frac{4}{(e^x + e^{-x})^2}$$



A função logística:

$$f(x) = \frac{1}{1 + e^{-x}} \quad f'(x) = \frac{df}{dx} = f(x)[1 - f(x)]$$



Perceptron Multicamadas & Funções Lógicas

- Aprendizagem supervisionada: altera a intensidade das conexões de maneira a diminuir o erro. Para isso, é apresentada a resposta à rede que ela deve aprender a responder frente ao estímulo.
- Isso é feito para todas as camadas, a de saída e as intermediárias.
- Mas a estimativa do sinal de erro difere para as camadas.

Perceptron Multicamadas & Funções Lógicas

Para as células da camada de saída:

$$e_k = (t_k - o_k)$$

Calculo do erro

$$\begin{aligned}\delta_{saída,k} &= f'(Zh_k) \odot (e_k) \\ &= o_k \odot (1 - o_k) \odot (t_k - o_k)\end{aligned}$$

Sinal do erro

- Correção da matriz de conexões Z

Na etapa $t+1$:
$$Z_{[t+1]} = Z_{\{t\}} + \eta \delta_{saída,k} h_k^T = Z_{\{t\}} + \Delta Z_t$$

Perceptron Multicamadas & Funções Lógicas

Para as células da camada intermediária:

Para as células intermediárias o sinal de erro não é comparado a partir do valor ideal ($e_k = t_k - o_k$). Ele é estimado a partir da retropropagação do sinal de erro da camada de saída e da ativação das células intermediárias ($Z_{[t]}^T \delta_{saída,k}$).

$$\begin{aligned}\delta_{intermediária,k} &= f'(Wx_k) \odot (Z_{[t]}^T \delta_{saída,k}) \\ &= h_k \odot (1 - h_k) \odot (Z_{[t]}^T \delta_{saída,k})\end{aligned}$$

Sinal do erro

- Correção da matriz de conexões W

Na etapa $t+1$:

$$\begin{aligned}W_{[t+1]} &= W_{[t]} + \eta \delta_{intermediária,k} x_k^T \\ &= W_{[t]} + \Delta W_t\end{aligned}$$

Perceptron Multicamadas & Funções Lógicas

- Um exemplo

Tomemos uma rede com $I = 3$ células de entrada, $L = 2$ células intermediárias e $J = 3$ células de saída.

$$W = \begin{bmatrix} 0,5 & 0,3 & 0,1 \\ 0,3 & 0,2 & 0,1 \end{bmatrix}$$

W associa as células da camada intermediária às de entrada. Notamos $L \times I = 2 \times 3$.

$$Z = \begin{bmatrix} 0,1 & 0,2 \\ 0,3 & 0,4 \\ 0,5 & 0,6 \end{bmatrix}$$

Z associa as células da camada de saída às intermediárias. Notamos $J \times L = 3 \times 2$.

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

Estímulo x .
Matriz 3×1 .

$$t = \begin{bmatrix} 0,1 \\ 0,3 \\ 0,7 \end{bmatrix}$$

Estímulo t . Matriz 3×1 .
Espera-se que a resposta seja a mais próxima possível de t .

Perceptron Multicamadas & Funções Lógicas

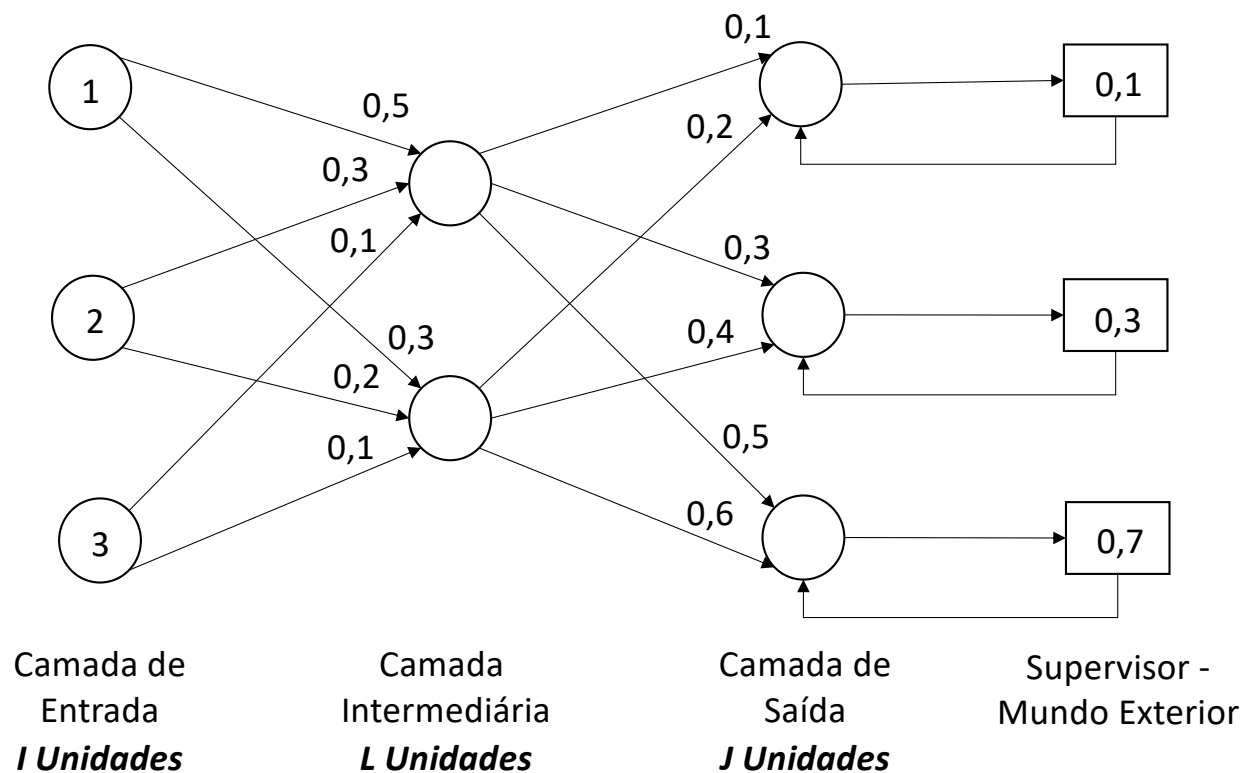


Figura: Rede de neurônios com camada intermediária que deve aprender a associar os estímulos x e t . Espera-se uma resposta o que seja a mais próxima de t . Neste sentido, visa-se minimizar o erro $e = (t-o)$.

Perceptron Multicamadas & Funções Lógicas

- Primeiramente no sentido normal (*feedforward*)

$$b = Wx = \begin{bmatrix} 1,4 \\ 1,0 \end{bmatrix} \quad \text{Ativação das células intermediárias}$$

$$h = f(b) = \begin{bmatrix} 0,8022 \\ 0,7311 \end{bmatrix} \quad \text{Resposta das células intermediárias}$$

$$a = Zh = \begin{bmatrix} 0,2264 \\ 0,5331 \\ 0,8397 \end{bmatrix} \quad \text{Ativação das células de saída}$$

$$o = f(a) = \begin{bmatrix} 0,5564 \\ 0,6302 \\ 0,6984 \end{bmatrix} \quad \text{Resposta das células de saída}$$

Perceptron Multicamadas & Funções Lógicas

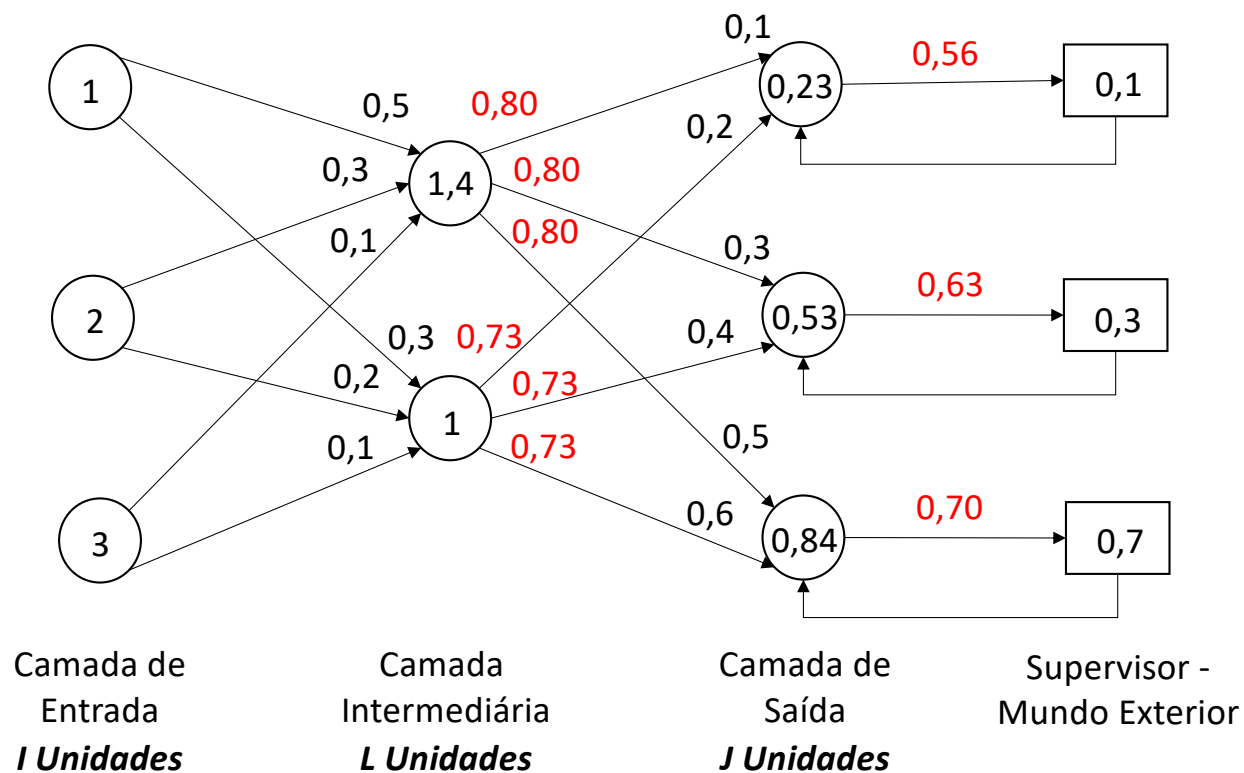


Figura: A ativação das células intermediárias é calculada por $b = Wx$. A ativação das células de saída por $a = Zh$. As respostas são calculadas aplicando a função logística. O erro é calculado pelo supervisor, $e = t - o$.

Perceptron Multicamadas & Funções Lógicas

- No sentido inverso
(*backpropagation*)

$$e = t - o = \begin{bmatrix} -0,4564 \\ -0,3302 \\ +0,0016 \end{bmatrix} \quad \text{O erro é calculado pelo supervisor}$$

$$f'(a) = o_k \odot (1 - o_k) \quad \text{Derivada do sinal de saída}$$

$$\delta_{saída} = f'(a) \odot e \quad \text{Sinal do erro da saída}$$

$$\delta_{saída} = o \odot (1 - o) \odot (t - o)$$

$$= \begin{bmatrix} -0,1126 \\ -0,0770 \\ +0,0003 \end{bmatrix}$$

Perceptron Multicamadas & Funções Lógicas

- No sentido inverso (*backpropagation*)

Sinal do erro
intermediário

$$\begin{aligned}\delta_{intermediária,k} &= f'(Wx_k) \odot (Z_{[t]}^T \delta_{saída,k}) \\ &= h_k \odot (1 - h_k) \odot (Z_{[t]}^T \delta_{saída,k})\end{aligned}$$

Estimação do erro pelas
células da camada
intermediária

$$f = R^T 1 = Z^T \delta_{saída}$$

Retorno

$$\begin{aligned}R &= Z \odot [\delta_{saída} \delta_{saída}] \\ &= \begin{bmatrix} 0,1 & 0,2 \\ 0,3 & 0,4 \\ 0,5 & 0,6 \end{bmatrix} \odot \begin{bmatrix} -0,1126 & -0,1126 \\ -0,0770 & -0,0770 \\ +0,0003 & +0,0003 \end{bmatrix} = \begin{bmatrix} -0,0113 & -0,0225 \\ -0,0231 & -0,0308 \\ +0,0002 & +0,0002 \end{bmatrix}\end{aligned}$$

Perceptron Multicamadas & Funções Lógicas

- No sentido inverso (*backpropagation*)

Estimação do erro
pelas células da
camada
intermediária

$$f = R^T 1 = Z^T \delta_{saída} = \begin{bmatrix} -0,0342 \\ -0,0531 \end{bmatrix}$$

Sinal do erro
intermediário

$$\begin{aligned} \delta_{intermediária,k} &= f'(Wx_k) \odot (Z_{[t]}^T \delta_{saída,k}) \\ \delta_{intermediária} &= h \odot (1 - h) \odot (Z_{[t]}^T \delta_{saída}) \\ &= \begin{bmatrix} -0,0054 \\ -0,0104 \end{bmatrix} \end{aligned}$$

Perceptron Multicamadas & Funções Lógicas

$$Z_{[t+1]} = Z_{\{t\}} + \Delta Z_t = Z_{\{t\}} + \eta \delta_{saída,k} h^T$$

- Correção da matriz Z

$$\Delta Z_t = \begin{bmatrix} -0,1126 \\ -0,0770 \\ +0,0003 \end{bmatrix} * [0,8022 \quad 0,7311]$$

$$Z_{[t+1]} = \begin{bmatrix} 0,1 & 0,2 \\ 0,3 & 0,4 \\ 0,5 & 0,6 \end{bmatrix} + \begin{bmatrix} -0,0904 & -0,0823 \\ -0,0617 & -0,0563 \\ +0,0003 & +0,0002 \end{bmatrix}$$

$$= \begin{bmatrix} 0,0096 & 0,1177 \\ 0,2383 & 0,3437 \\ 0,5003 & 0,6002 \end{bmatrix}$$

Perceptron Multicamadas & Funções Lógicas

$$W_{[t+1]} = W_{[t]} + \Delta W_t = W_{[t]} + \eta \delta_{intermediária,k} x^T$$

- Correção da matriz W

$$\Delta W_t = \begin{bmatrix} -0,0054 \\ -0,01104 \end{bmatrix} * [1 \quad 2 \quad 3]$$

$$W_{[t+1]} = \begin{bmatrix} 0,5 & 0,3 & 0,1 \\ 0,3 & 0,2 & 0,1 \end{bmatrix} + \begin{bmatrix} -0,0054 & -0,0108 & -0,0163 \\ -0,0104 & -0,0209 & -0,0313 \end{bmatrix}$$

$$= \begin{bmatrix} 0,4946 & 0,2892 & 0,0837 \\ 0,2896 & 0,1791 & 0,0687 \end{bmatrix}$$

Perceptron Multicamadas & Funções Lógicas

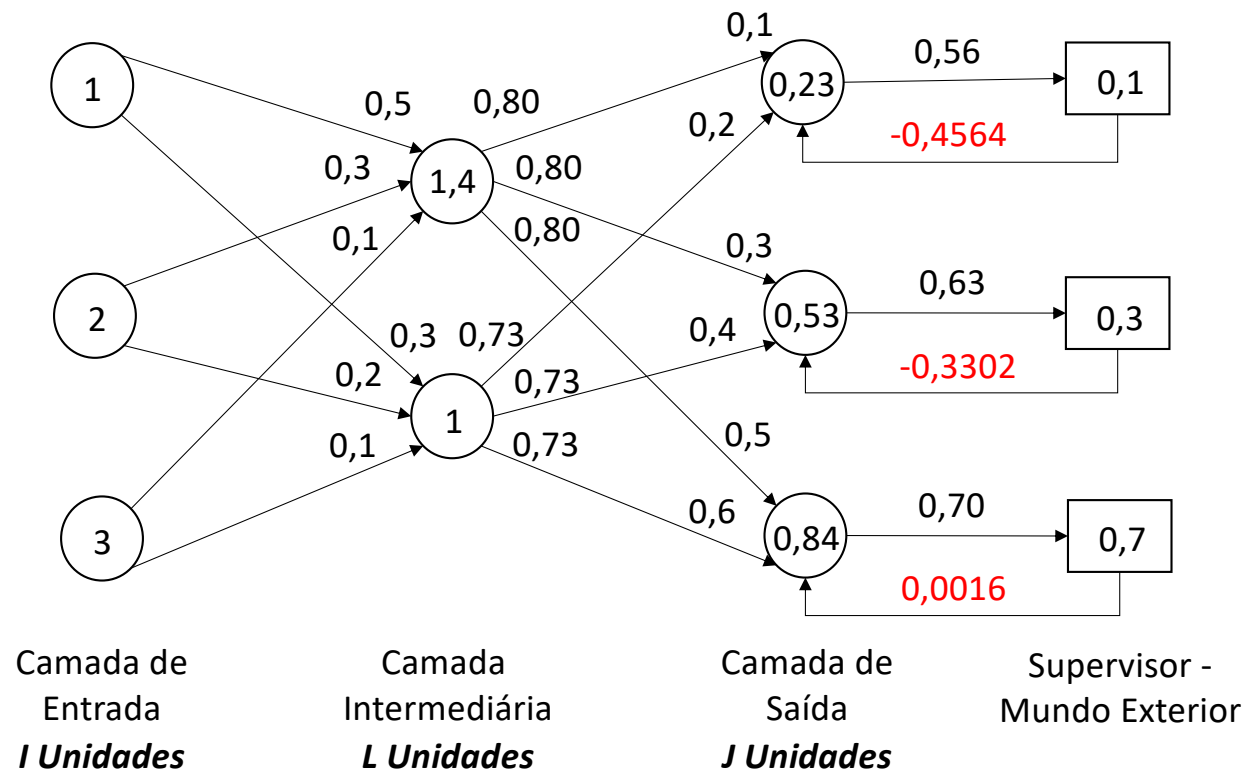


Figura: O erro é calculado pelo supervisor, $e = t - o$.

Perceptron Multicamadas & Funções Lógicas

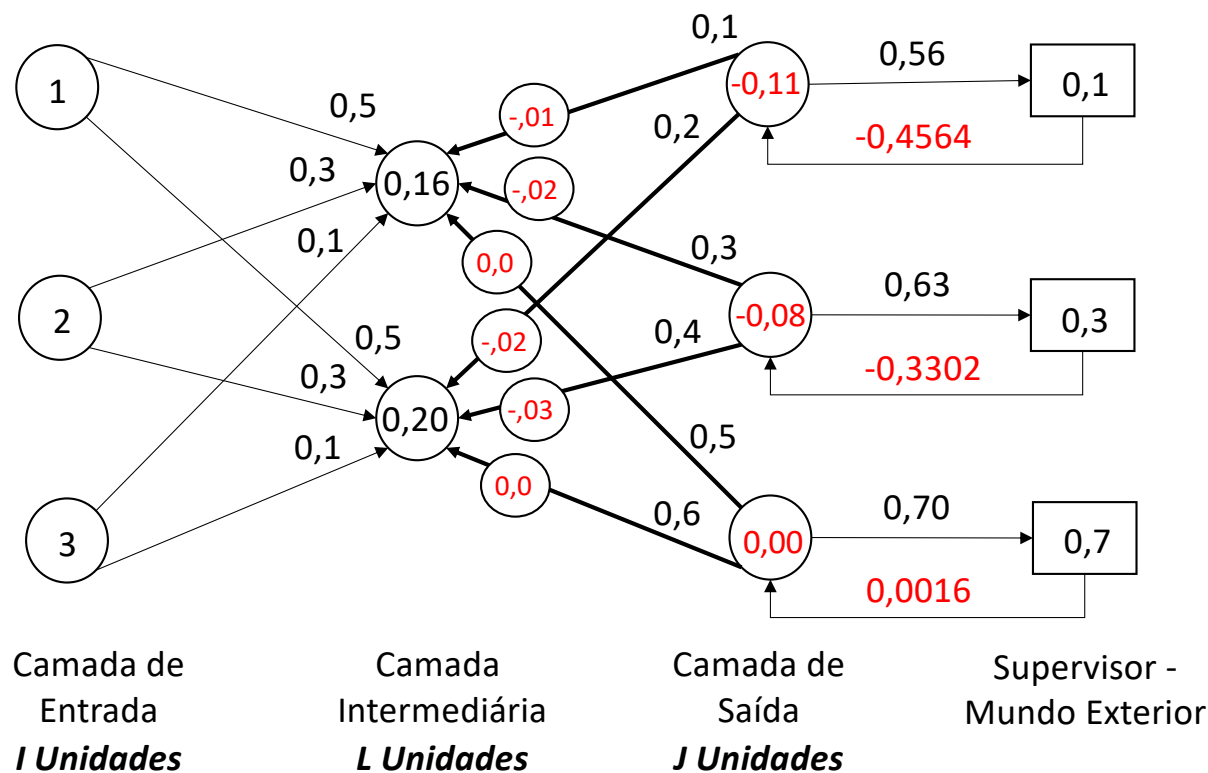


Figura: As células da camada de saída calculam o sinal do erro $\delta_{saída} = o \odot (1 - o) \odot e$. O sinal de erro $\delta_{saída}$ é multiplicado pelas intensidades das sinapses Z. $R = Z \odot [\delta_{saída} \delta_{saída}]$ Um retorno R é enviado para as células da camada intermediária.

Perceptron Multicamadas & Funções Lógicas

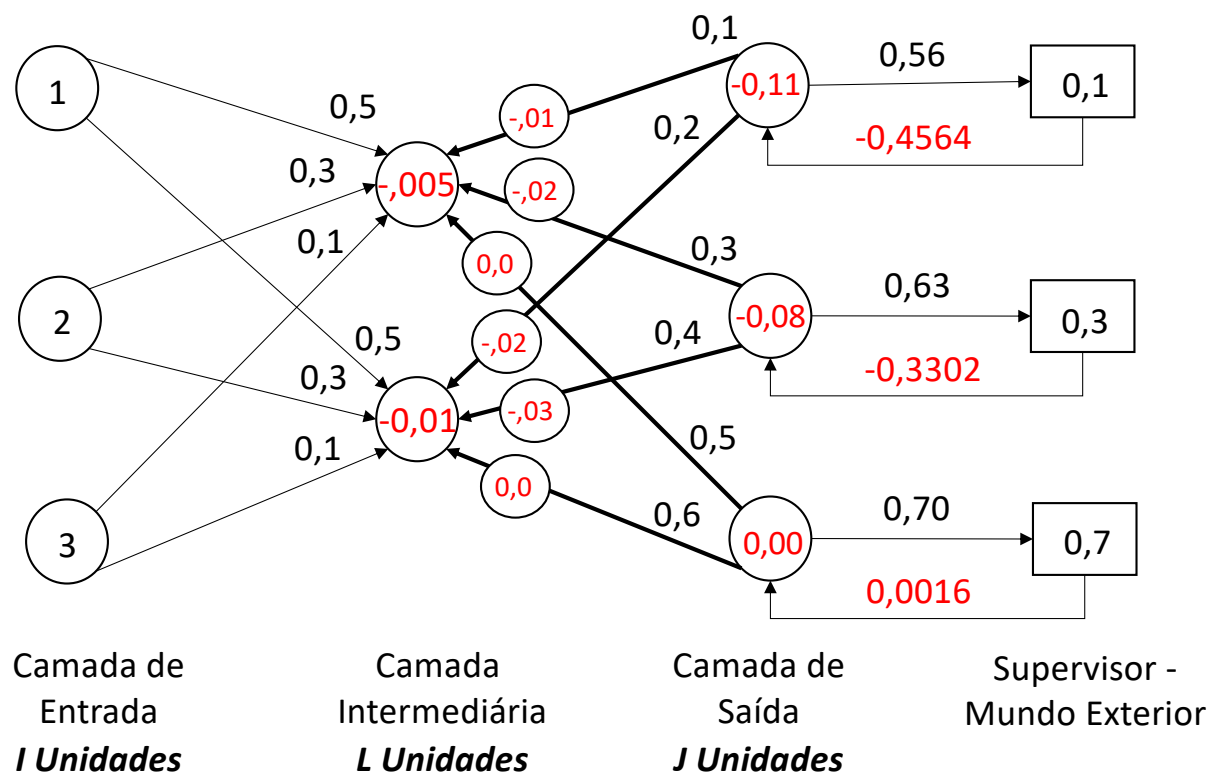


Figura: As células da camada intermediária calculam o sinal do erro

$$\delta_{intermediária,k} = f'(Wx) \odot R^T 1.$$

As células da camada de saída corrigem Z . $\eta=1$.

$$Z_{[t+1]} = Z_{[t]} + \eta \delta_{saída,k} h_k^T$$

Perceptron Multicamadas & Funções Lógicas

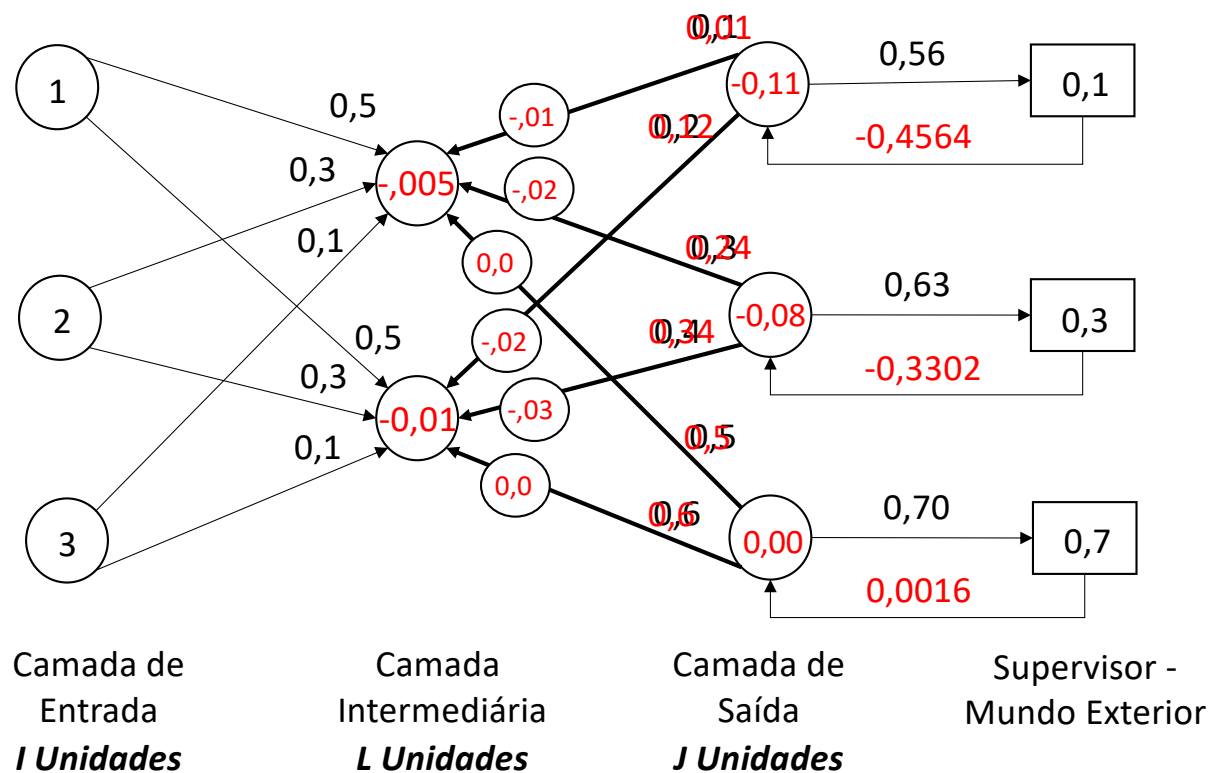


Figura: As células da camada intermediária calculam o sinal do erro

$$\delta_{intermediária,k} = f'(Wx) \odot R^T 1.$$

As células da camada de saída e intermediária corrigem Z. $\eta=1$. $Z_{[t+1]} =$

$$Z_{[t]} + \eta \delta_{saída} h^T.$$

As células da camada de entrada e intermediária corrigem W. $\eta=1$. $W_{[t+1]} =$

$$W_{[t]} + \eta \delta_{intermediária} x^T.$$

Perceptron Multicamadas & Funções Lógicas

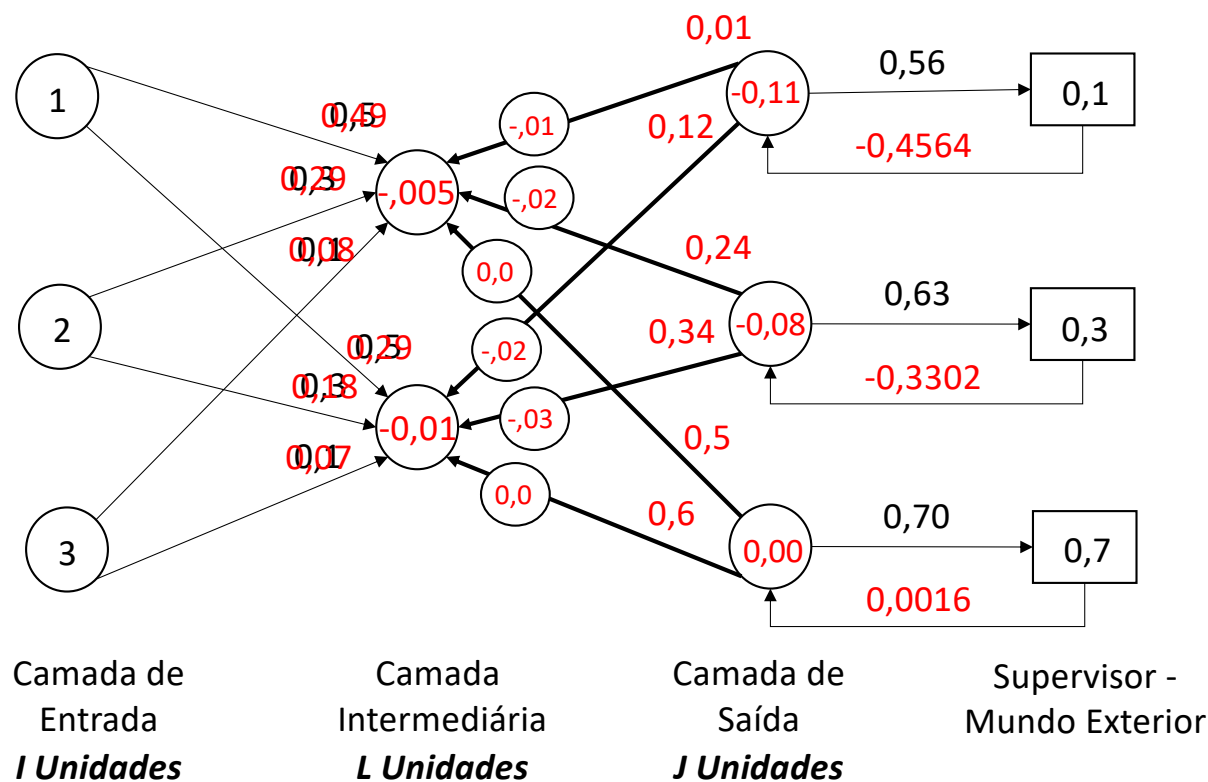


Figura: As células da camada intermediária calculam o sinal do erro $\delta_{intermediária} = f'(Wx) \odot R^T 1$.

As células da camada de saída e intermediária corrigem Z . $\eta=1$. $Z_{[t+1]} = Z_{[t]} + \eta \delta_{saída} h^T$.

As células da camada de entrada e intermediária corrigem W . $\eta=1$. $W_{[t+1]} = W_{[t]} + \eta \delta_{intermediária} x^T$.

Perceptron Multicamadas & Funções Lógicas

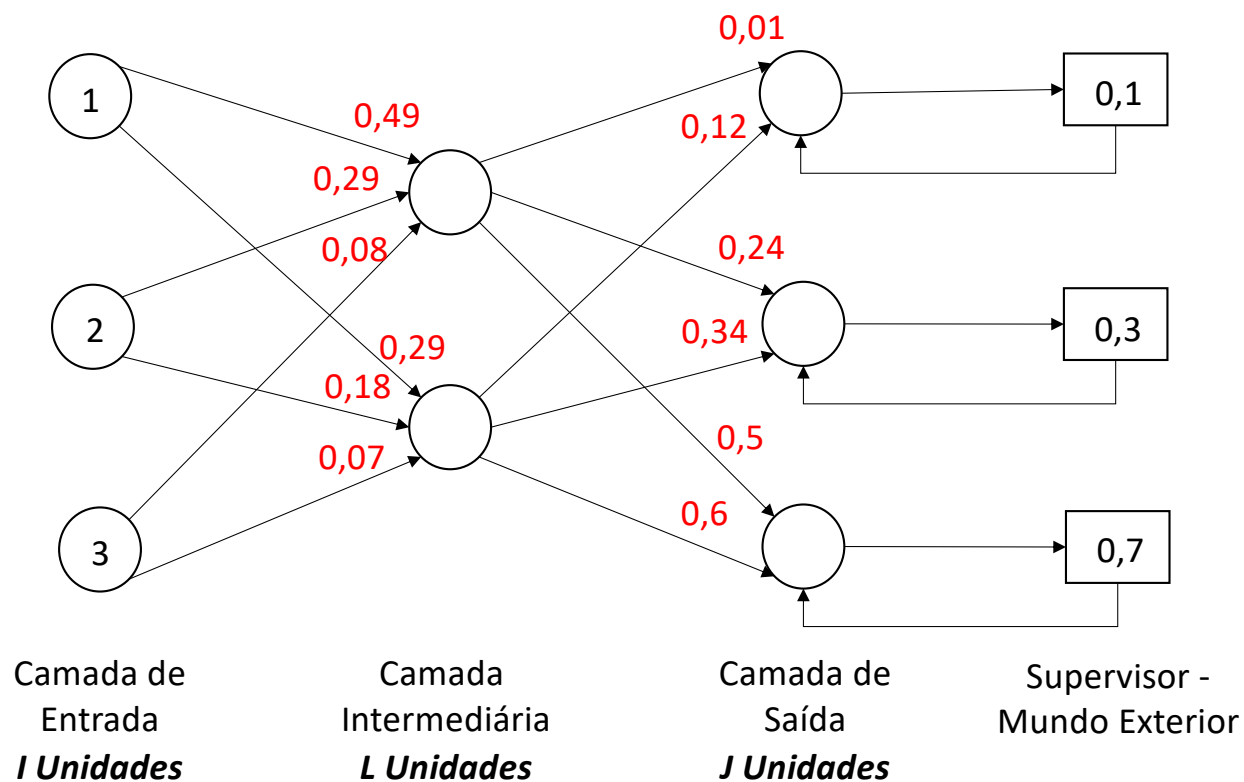


Figura: Um outro ciclo de aprendizagem pode começar:

$$b = Wx$$

$$h = f(b)$$

$$a = Zh$$

$$o = f(a)$$

$$e = t - o$$

$$\delta_{saída}$$

$$\delta_{intermediária}$$

$$Z_{[t+1]} = Z_{[t]} + \Delta Z_t$$

$$W_{[t+1]} = W_{[t]} + \Delta W_t$$

Perceptron Multicamadas & Funções Lógicas

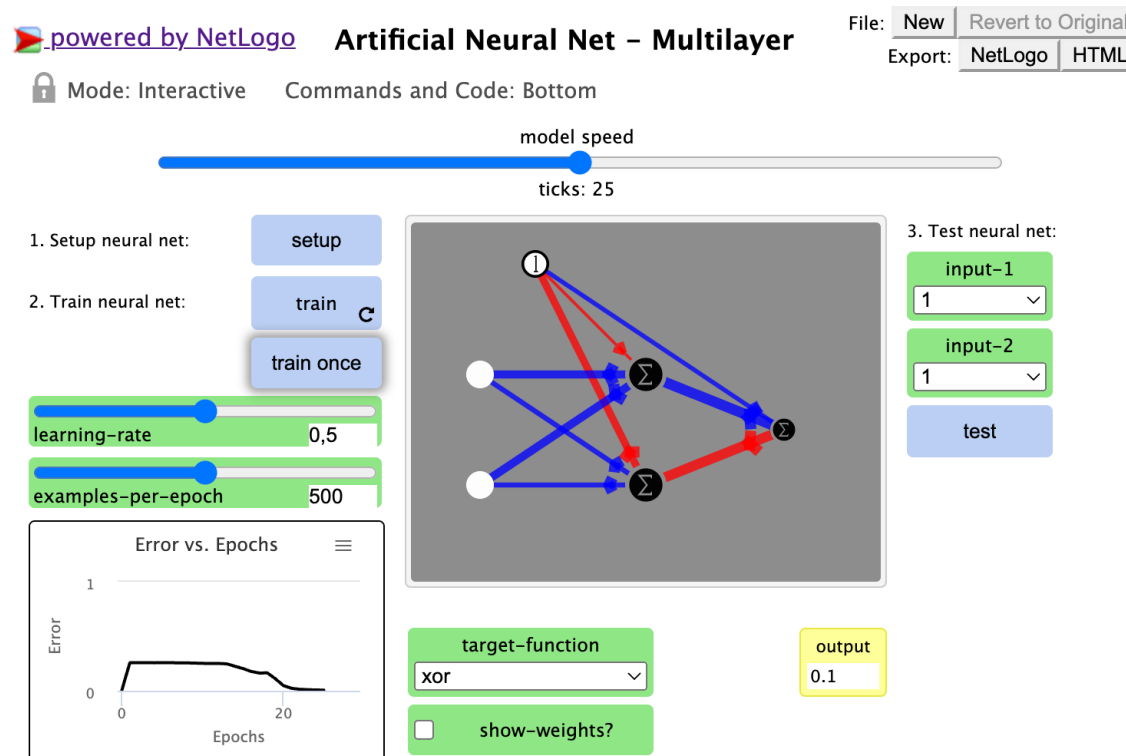


Figura: Uma rede neural artificial com uma camada intermediária de duas células, uma célula de saída e duas de entrada, implementada no NetLogo® para aprendizagem de funções lógicas.

Fonte: <http://www.netlogoweb.org/launch#http://ccl.northwestern.edu/netlogo/models/models>

Perceptron Multicamadas & Funções Lógicas

Exercício

- Conceber e implementar em C++ um perceptron multicamadas para a aprendizagem das funções lógicas OU, E e XOU.

Tabela: Conectivos lógicos

a	b	a OU b	a E b	a XOU b
1	1	1	1	0
1	0	1	0	1
0	1	1	0	1
0	0	0	0	0

Referências

- Abdi, H. **Les réseaux de neurones**. Grenoble: Presses Universitaires de Grenoble, 1993, p. 273.
- Abdi, H.; Valentin, D. **Mathématiques pour les Sciences Cognitives avec des applications aux réseaux de neurones, au traitement du signal, à l'imagerie cérébrale et à la statistique**. Grenoble: Presses Universitaires de Grenoble, 2006, p. 357.
- Data Science Academy. **Deep Learning Book**, 2022. Disponível em: [/www.deeplearningbook.com.br/](http://www.deeplearningbook.com.br/)>. Acesso em: 09 Junho. 2023.
- Rand, W.; Wilensky, U. NetLogo Artificial Neural Net - Multilayer model. <http://ccl.northwestern.edu/netlogo/models/ArtificialNeuralNet-Multilayer>. **Center for Connected Learning and Computer-Based Modeling**, Northwestern University, Evanston, IL, 2006.